

Clustering with user constraints

Dimitrios Gunopulos

Clustering Data

- **The clustering problem:**

Given a set of objects, find groups of similar objects

- **Cluster:** a collection of data objects

- Similar to one another within the same cluster
- Dissimilar to the objects in other clusters

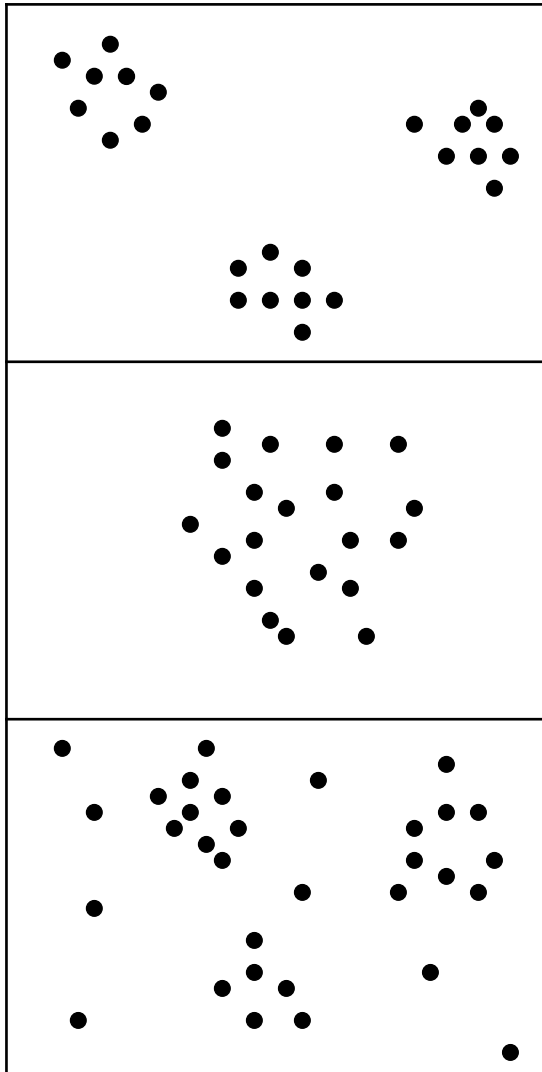
- **What is similar?**

Define appropriate metrics

- **Applications in**

- marketing, image processing, biology

Clustering



Sometimes easy

Sometimes impossible

and sometimes in between

Clustering Methods

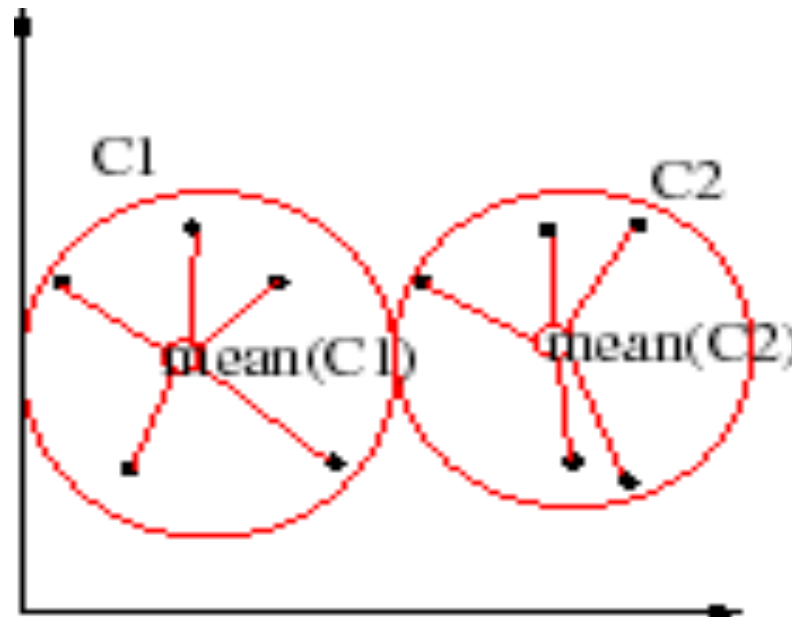
- **K-Means and K-medoids algorithms**
 - PAM, CLARA, CLARANS [Ng and Han, VLDB 1994]
- **Hierarchical algorithms**
 - CURE [Guha et al, SIGMOD 1998]
 - BIRCH [Zhang et al, SIGMOD 1996]
 - CHAMELEON [IEEE Computer, 1999]
- **Density based algorithms**
 - DENCLUE [Hinneburg, Keim, KDD 1998]
 - DBSCAN [Ester et al, KDD 96]
- **Subspace Clustering**
 - CLIQUE [Agrawal et al, SIGMOD 1998]
 - PROCLUS [Agrawal et al, SIGMOD 1999]
 - ORCLUS: [Aggarwal, and Yu, SIGMOD 2000]
 - DOC: [Procopiuc, Jones, Agarwal, and Murali, SIGMOD, 2002]

K-Means and K-Medoids algorithms

- Minimizes the sum of square distances of points to cluster representative

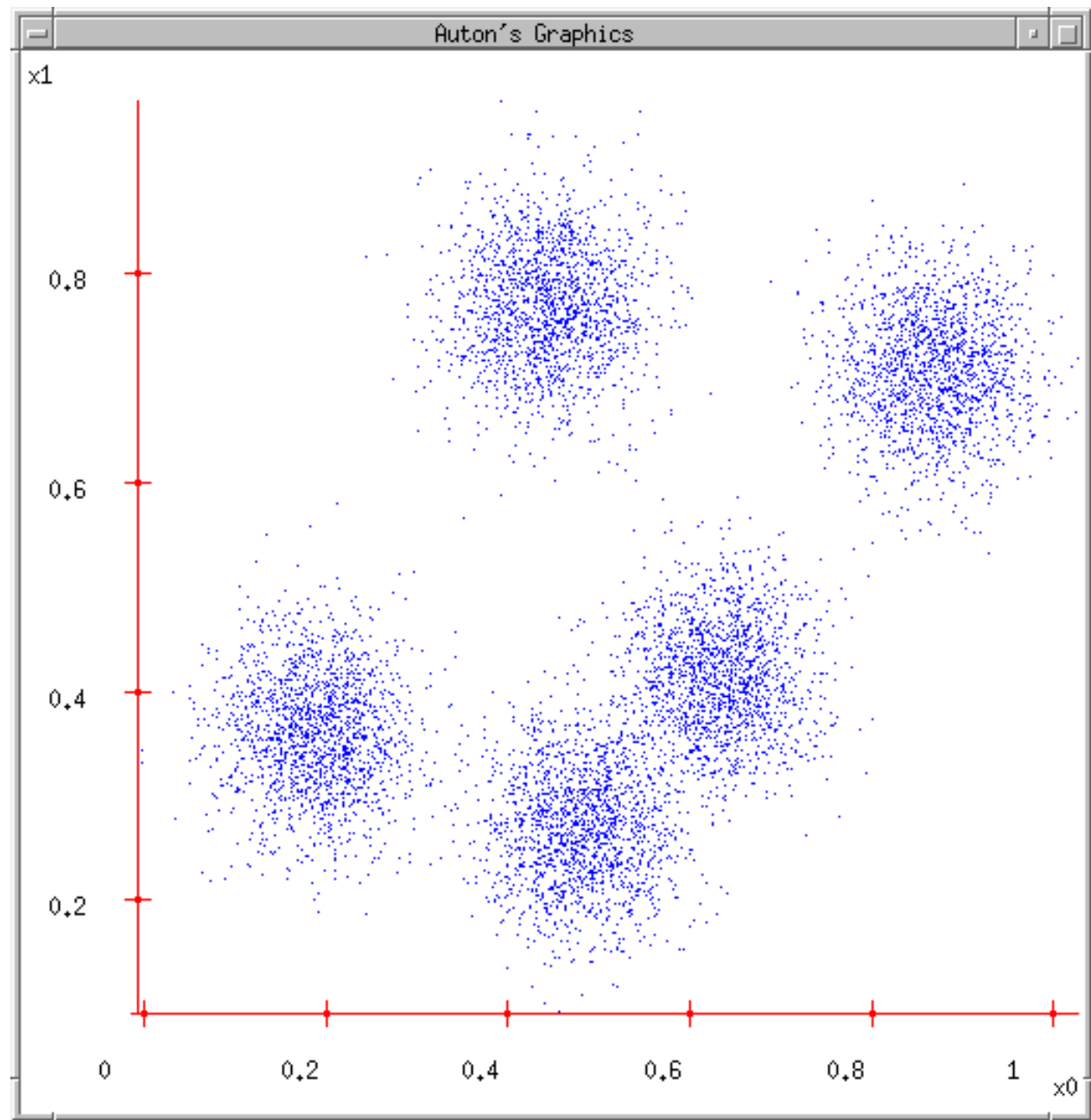
$$E_K = \sum_k \|x_k - m_{c(x_k)}\|^2$$

- Efficient iterative algorithms ($O(n)$)



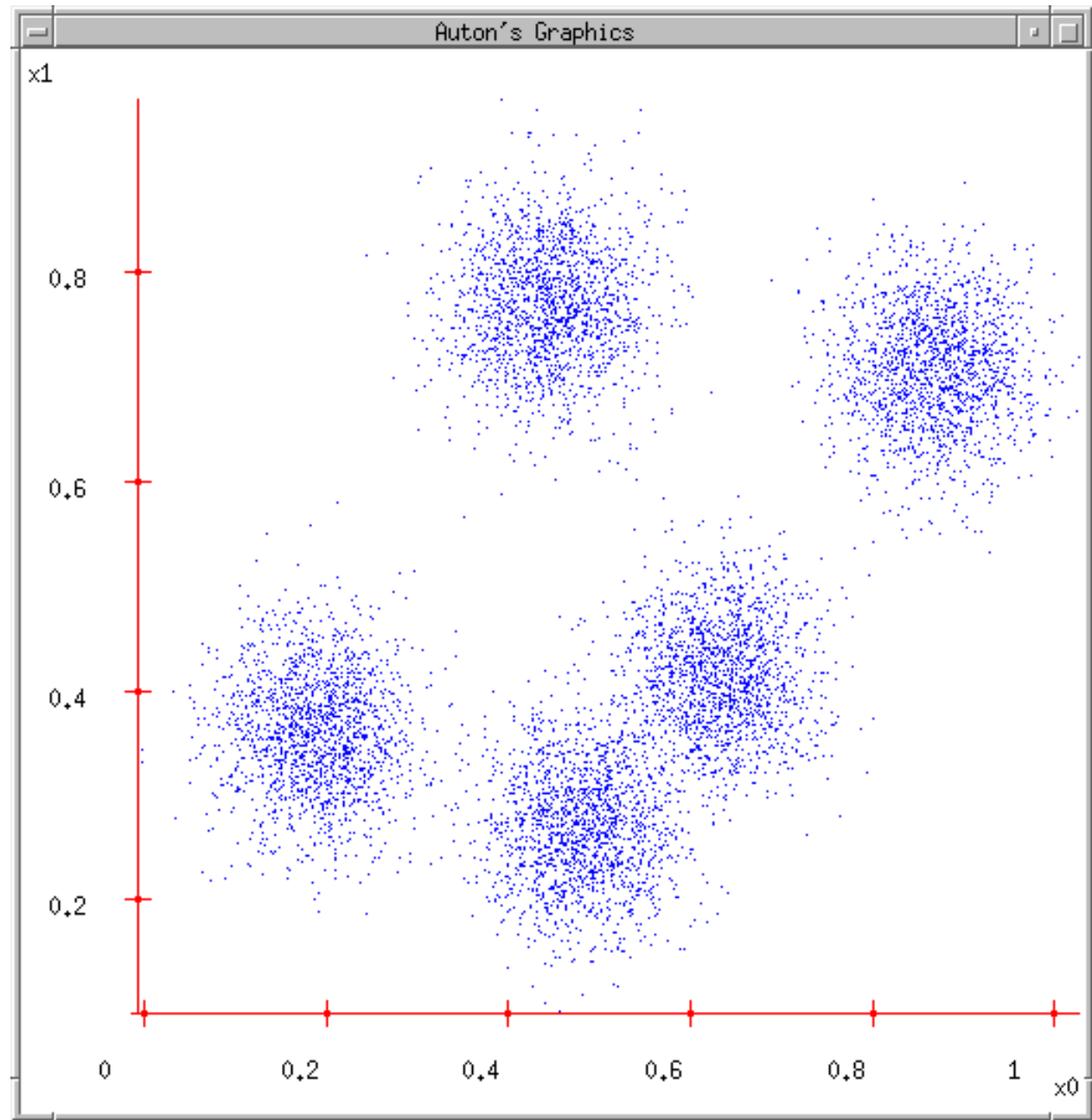
K-means example

(courtesy of
Andrew Moore, CMU)



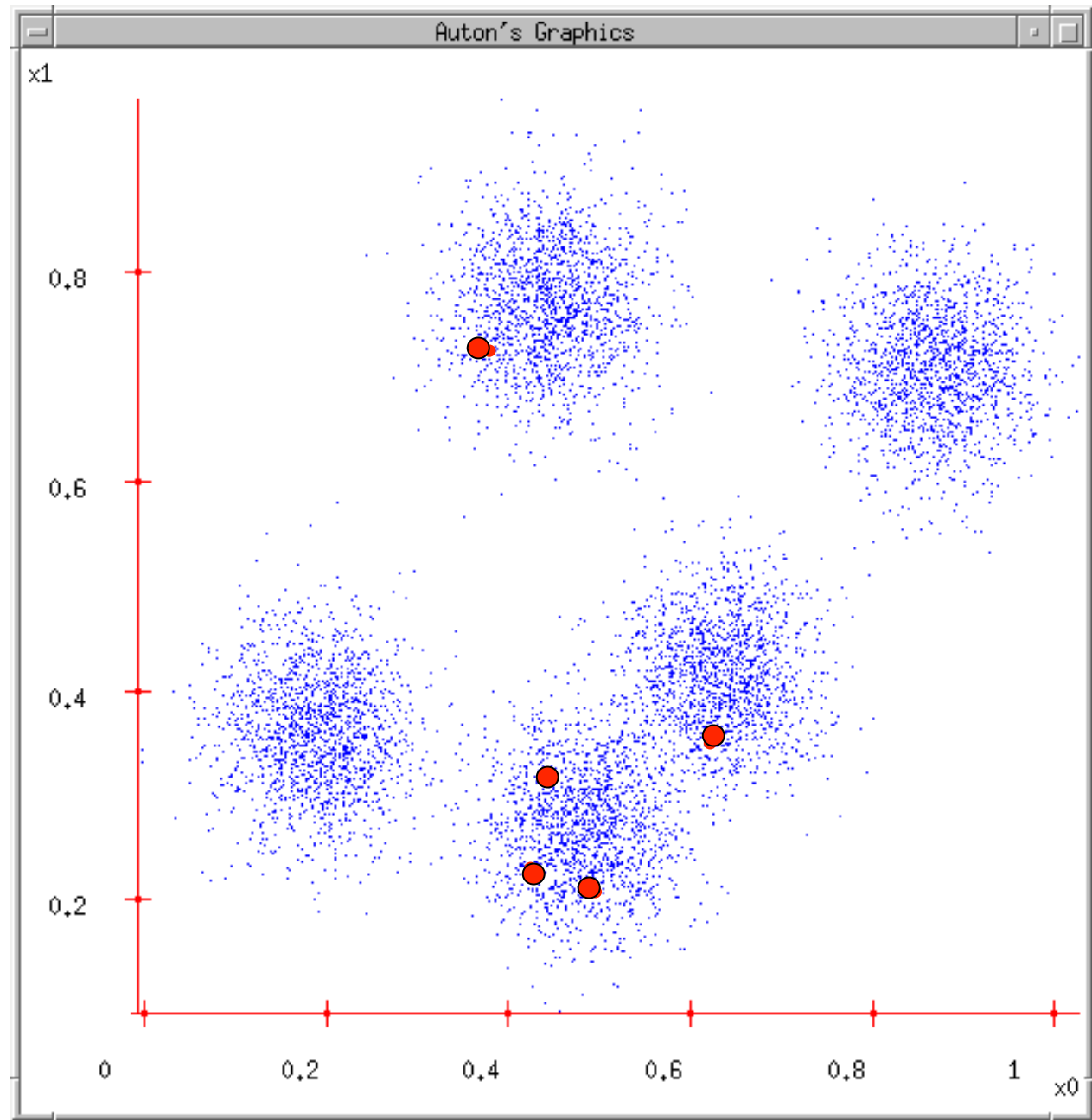
K-means

1. Ask user how many clusters they'd like.
(e.g. $K=5$)



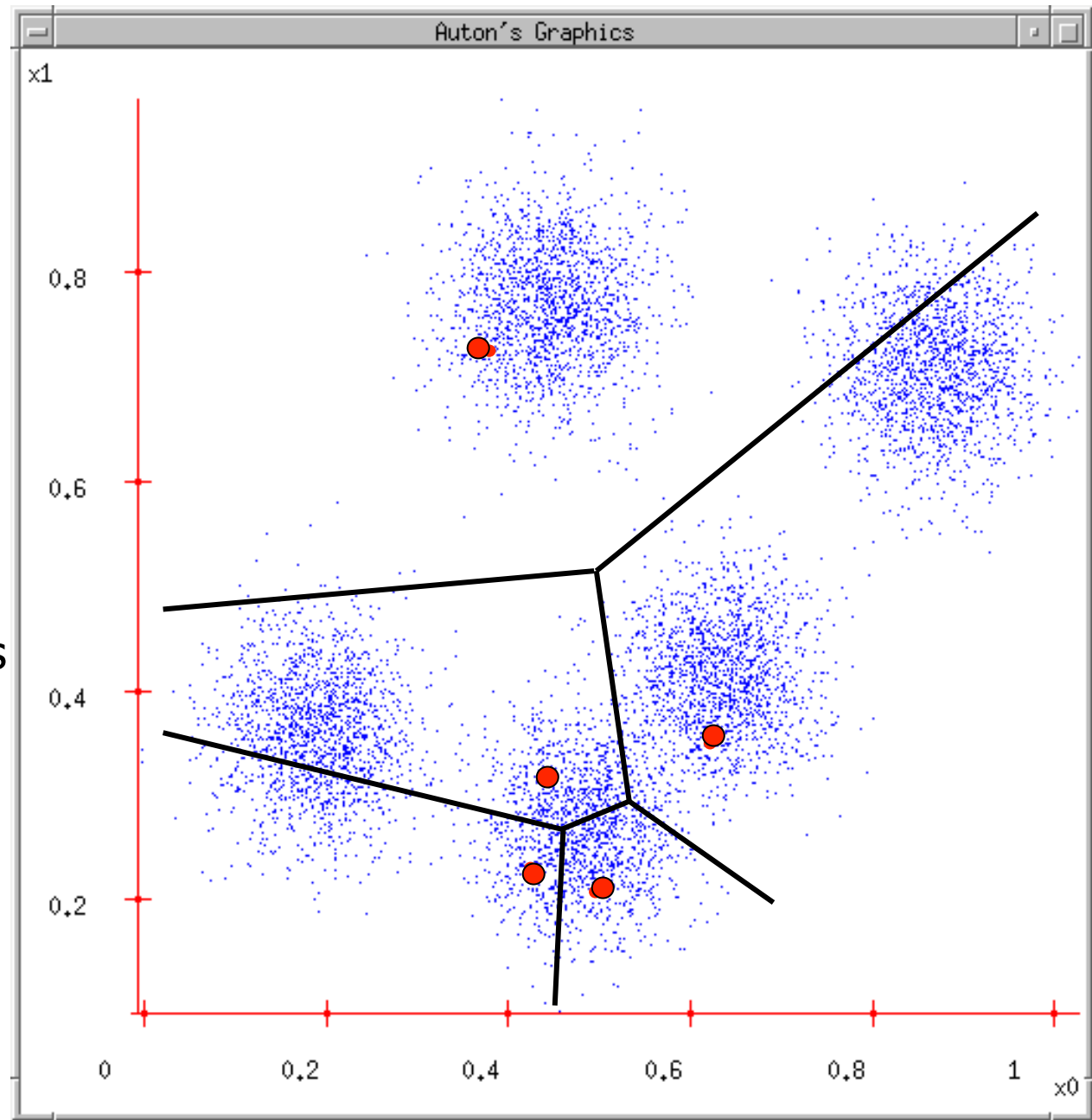
K-means

1. Ask user how many clusters they'd like.
(e.g. $K=5$)
2. Randomly guess K cluster Center locations



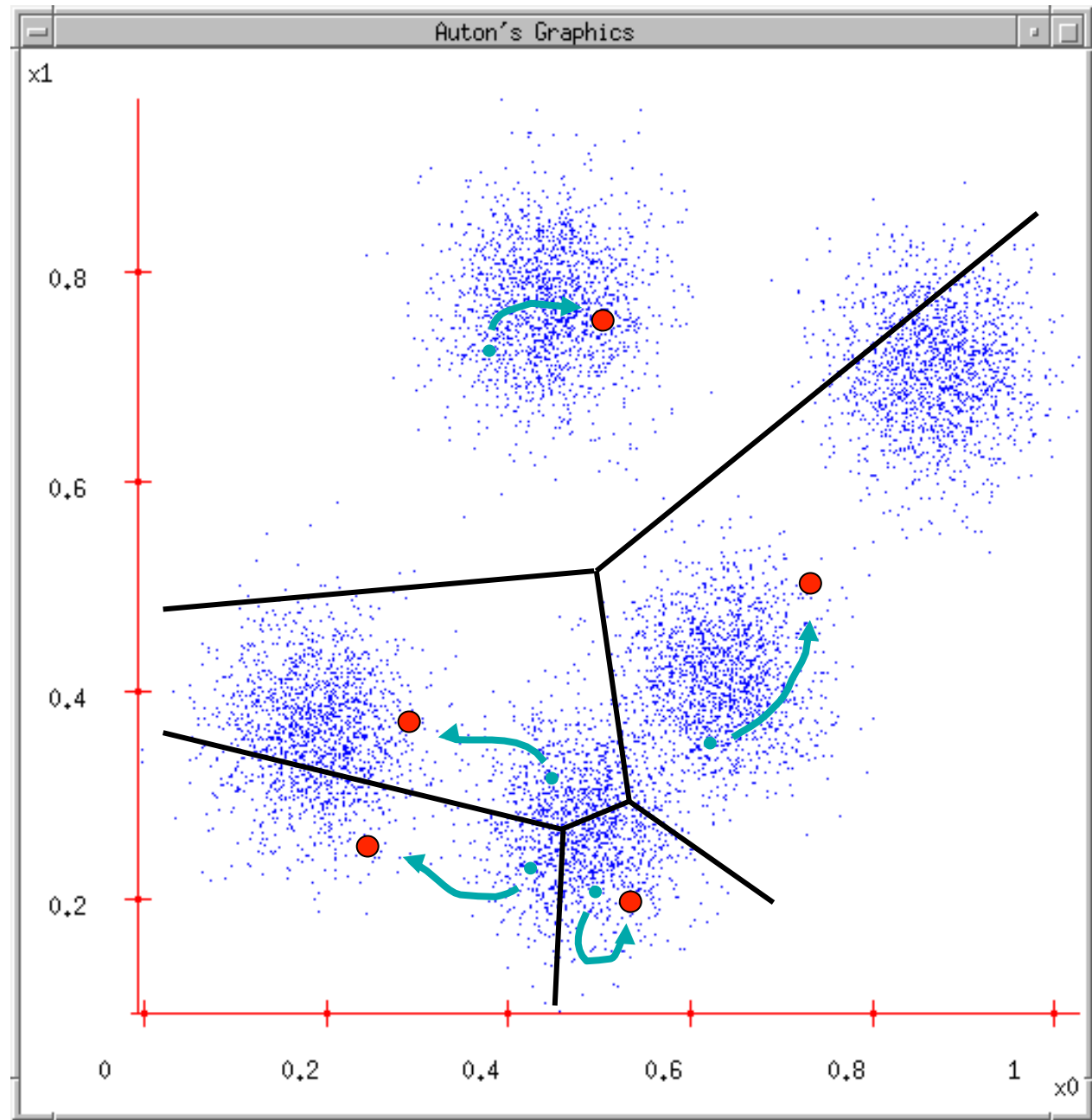
K-means

1. Ask user how many clusters they'd like.
(e.g. $K=5$)
2. Randomly guess K cluster Center locations
3. Each datapoint finds out which Center it's closest to. (Thus each Center "owns" a set of datapoints)



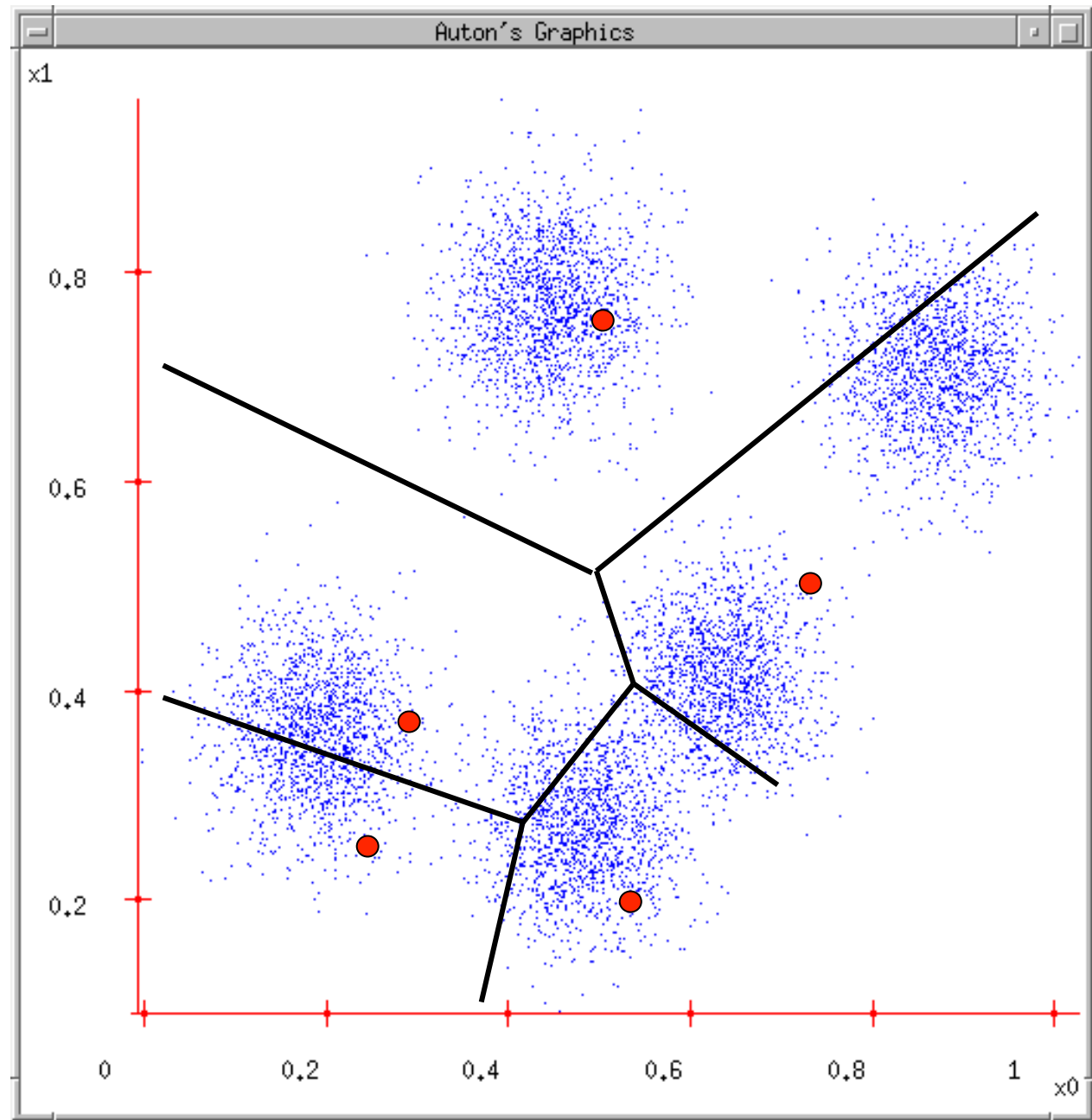
K-means

1. Ask user how many clusters they'd like.
(e.g. $k=5$)
2. Randomly guess k cluster Center locations
3. Each datapoint finds out which Center it's closest to.
4. Each Center finds the centroid of the points it owns



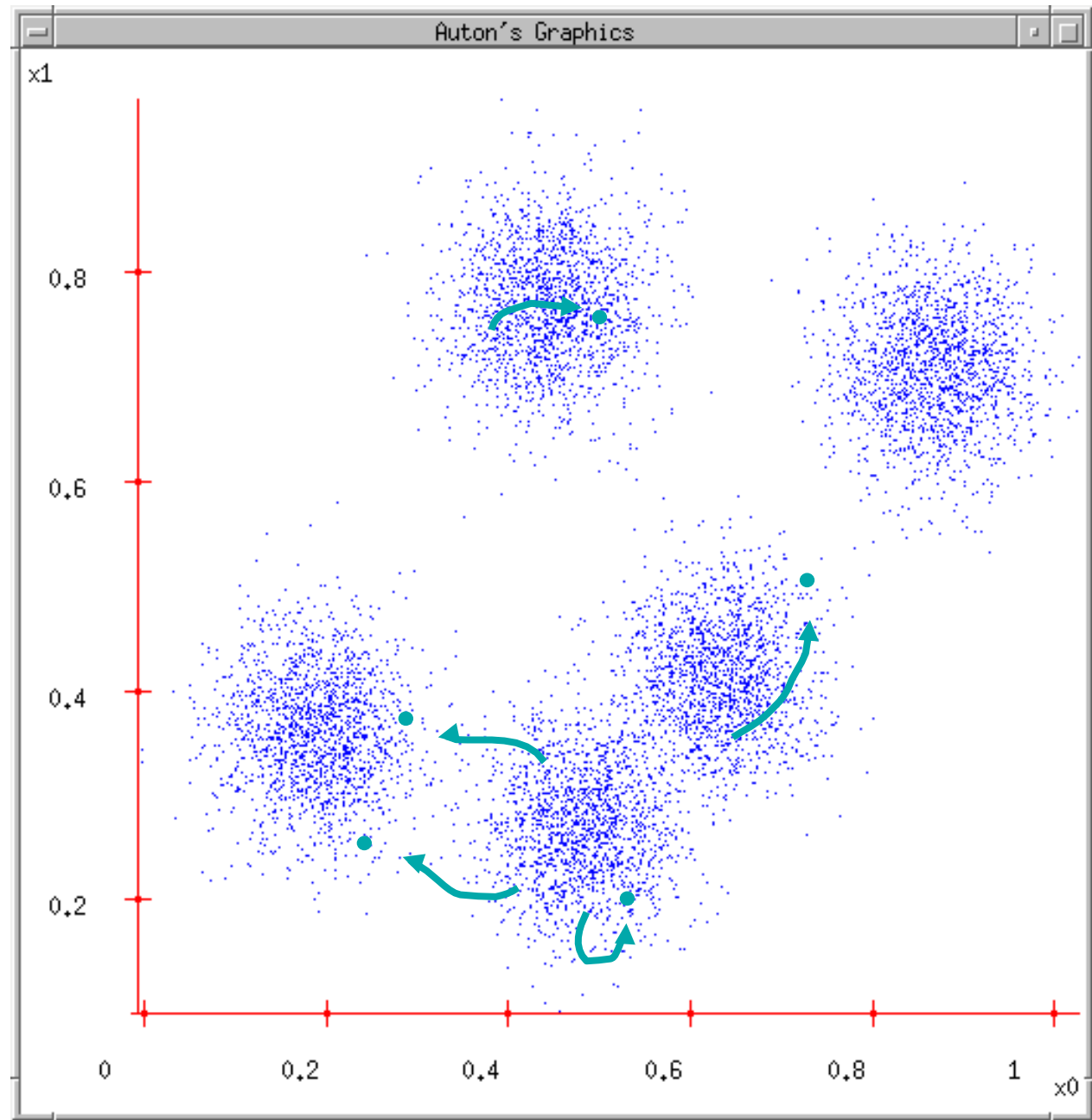
K-means

1. Ask user how many clusters they'd like.
(e.g. $k=5$)
2. Randomly guess k cluster Center locations
3. Each datapoint finds out which Center it's closest to.
4. Each Center finds the centroid of the points it owns
5. New Centers => new boundaries
6. Repeat until no change!



K-means

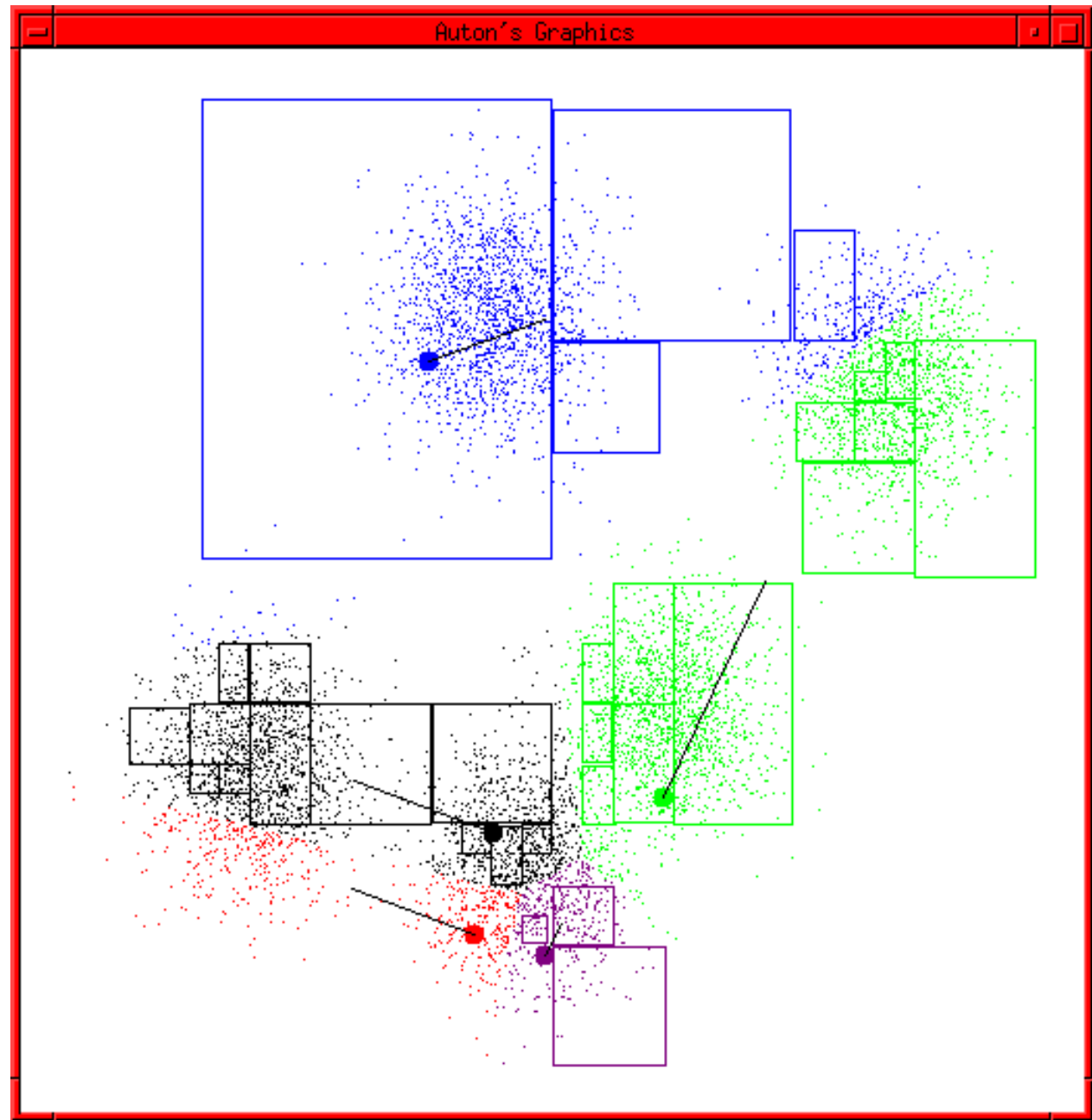
1. Ask user how many clusters they'd like.
(e.g. $k=5$)
2. Randomly guess k cluster Center locations
3. Each datapoint finds out which Center it's closest to.
4. Each Center finds the centroid of the points it owns...
5. ...and jumps there
6. ...Repeat until terminated!



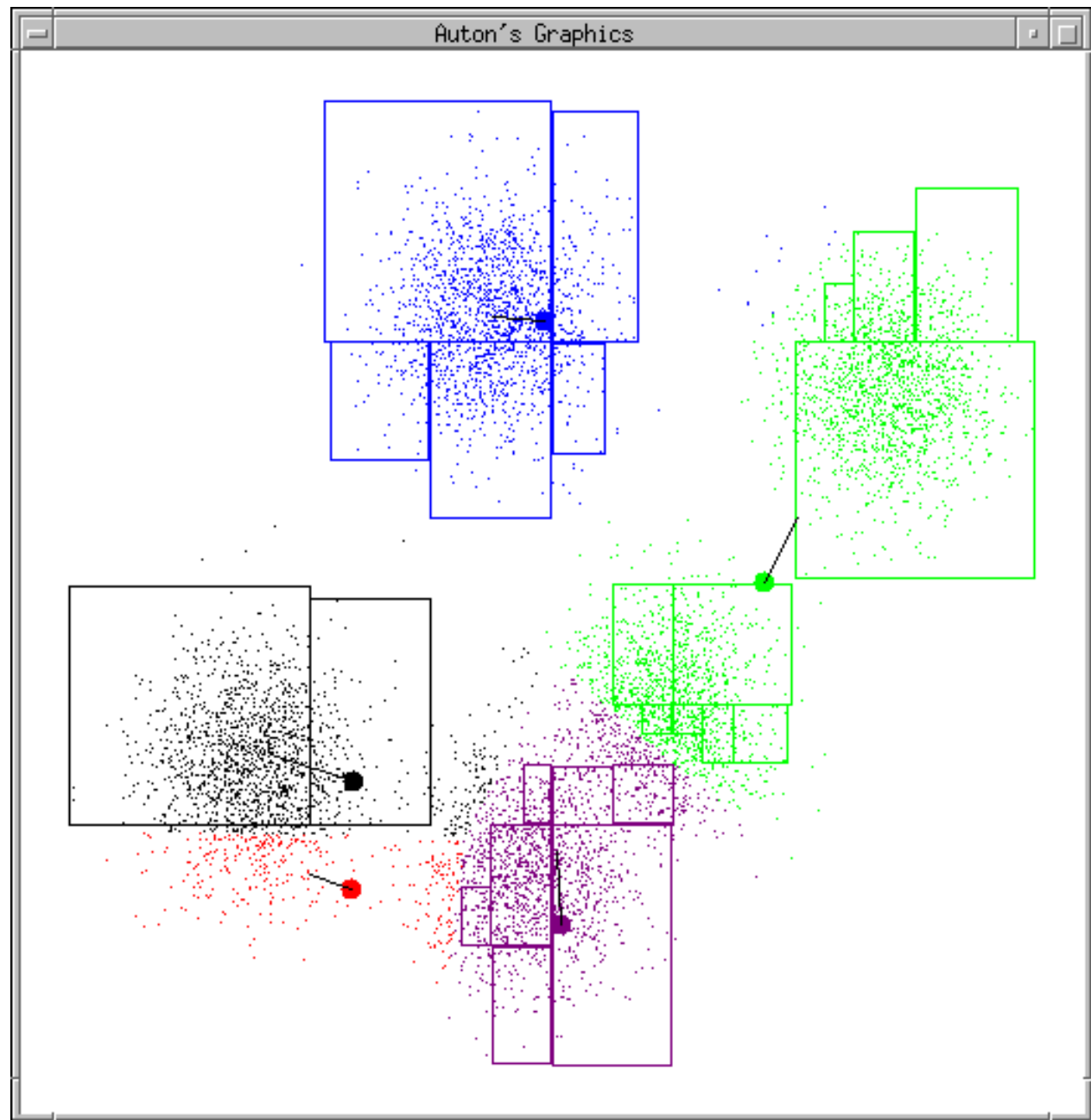
Accelerated Computations

Example generated by
Pelleg and Moore's
accelerated k-means

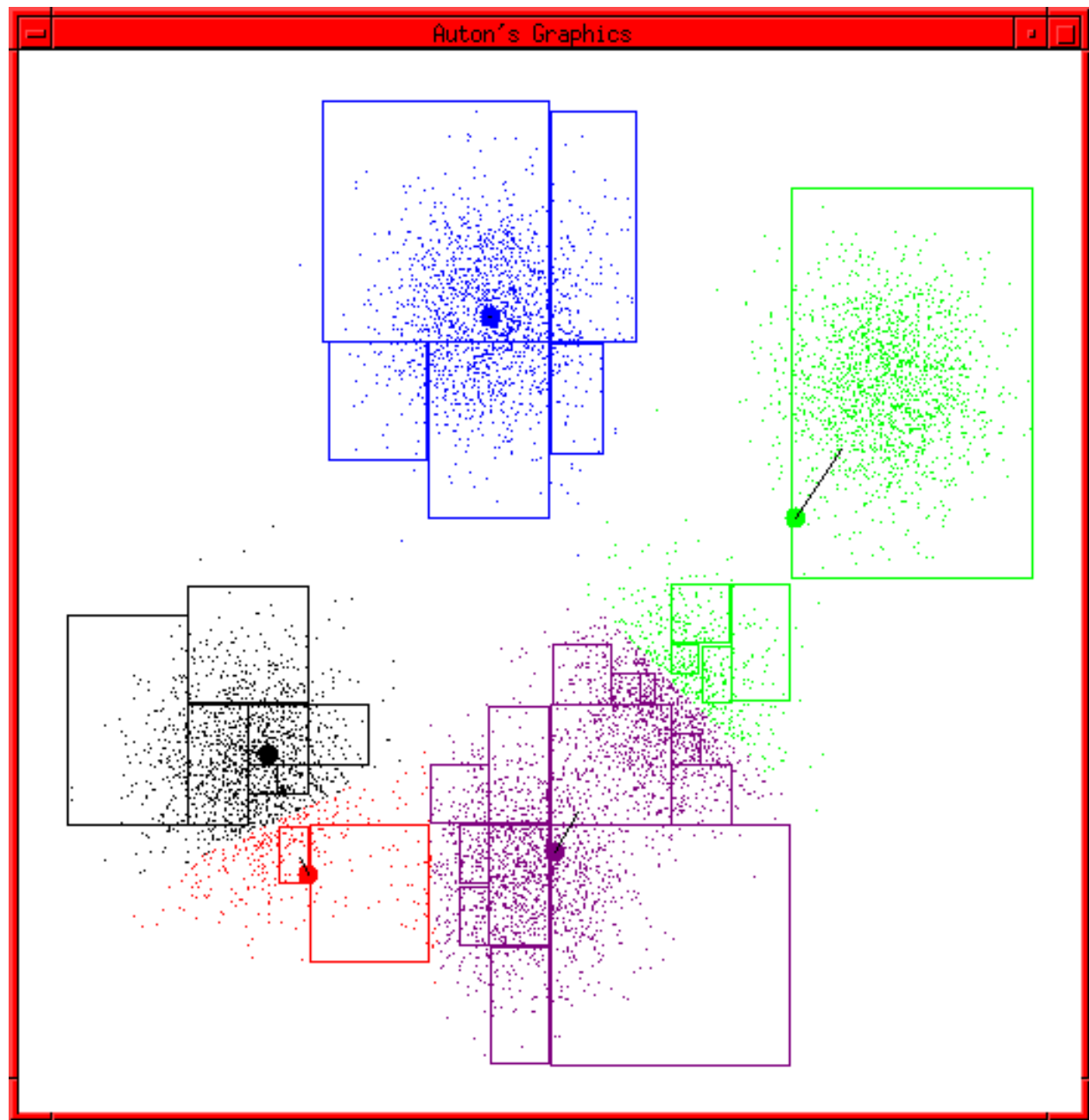
*Dan Pelleg and Andrew
Moore. Accelerating Exact
k-means Algorithms with
Geometric Reasoning.
Proc. Conference on
Knowledge Discovery in
Databases 1999, (KDD99)
(available on
www.autonlab.org/pap.html)*



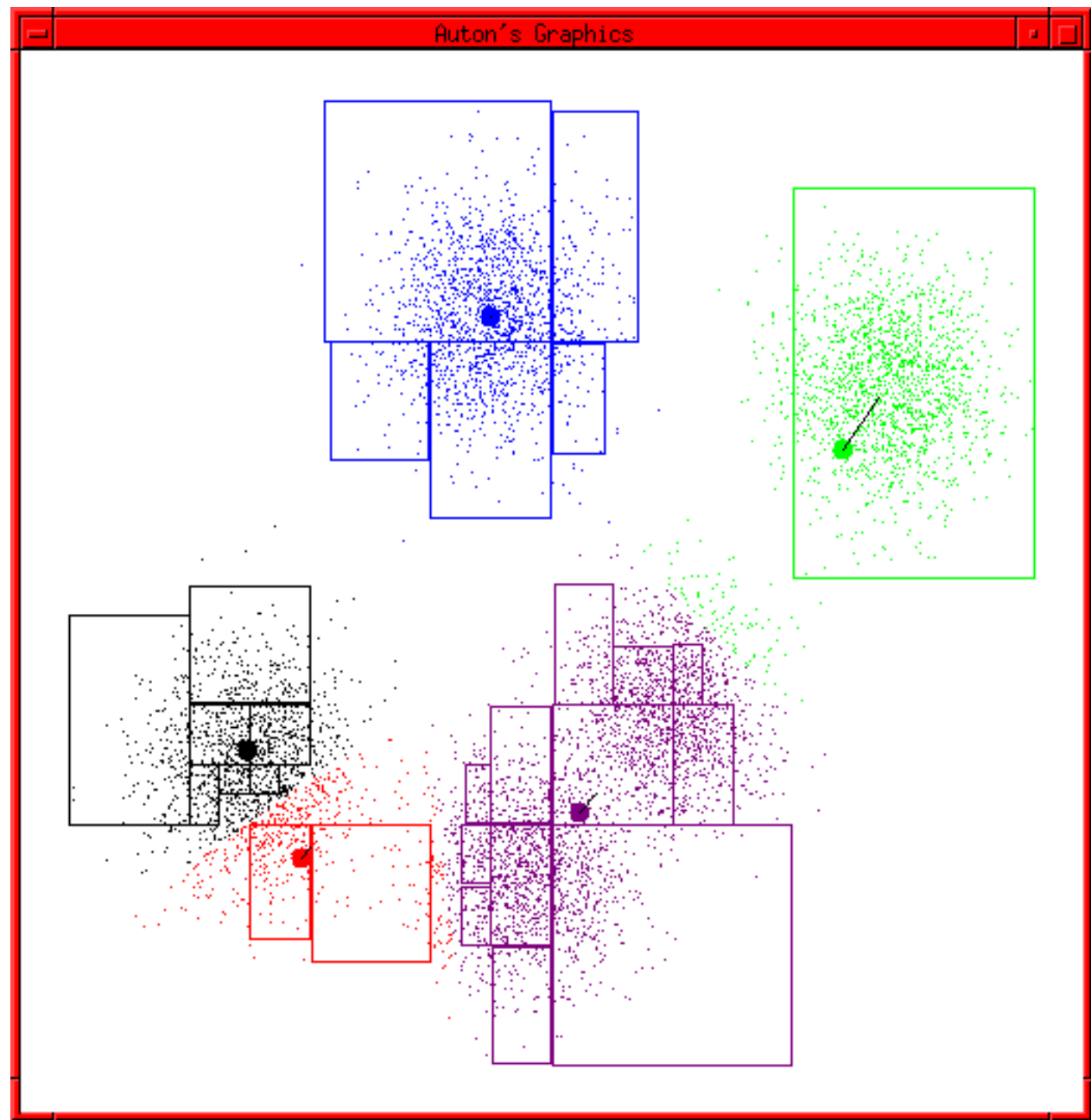
K-means
continues...



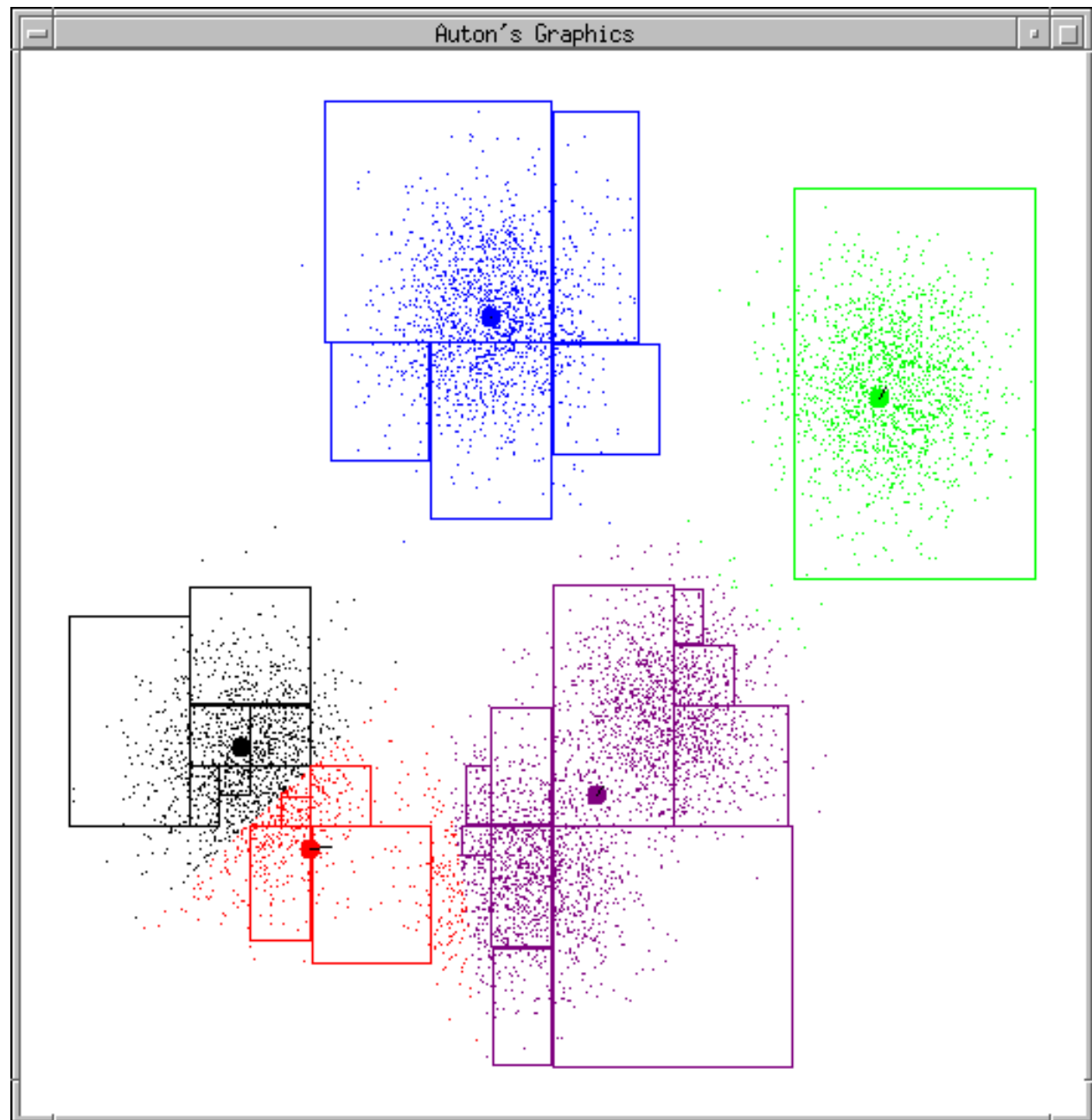
K-means
continues...



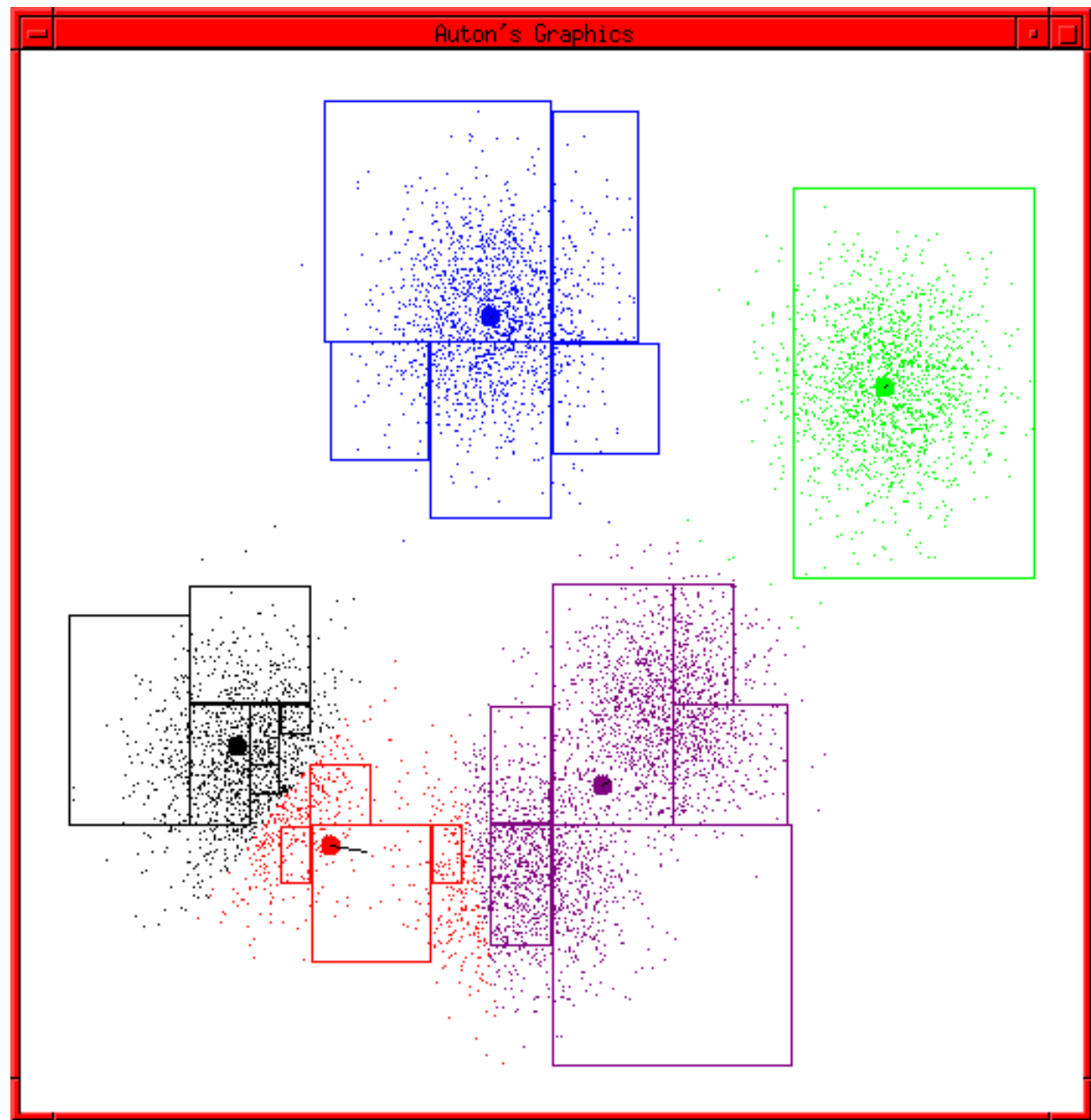
K-means
continues...



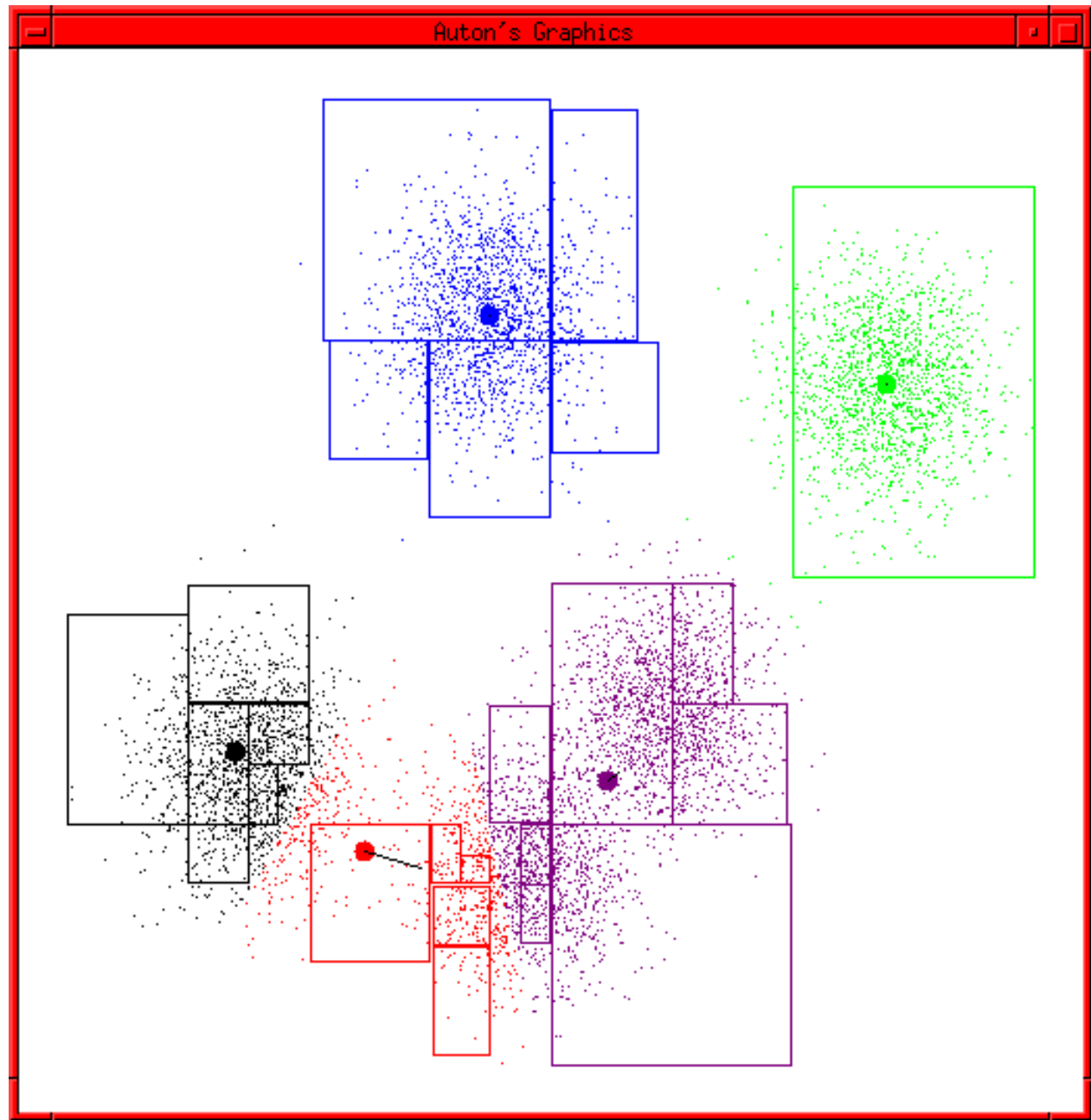
K-means
continues...



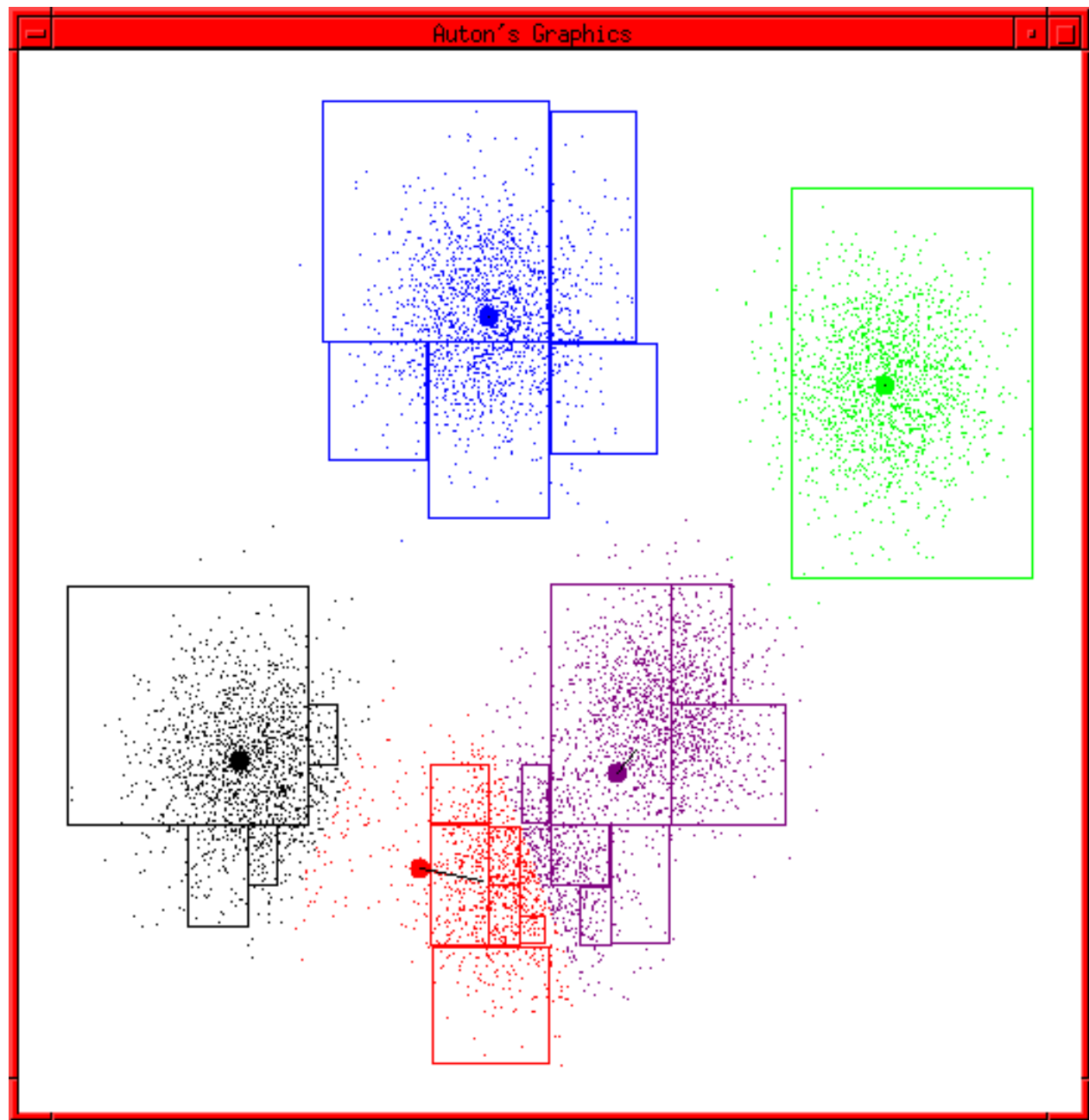
K-means
continues...



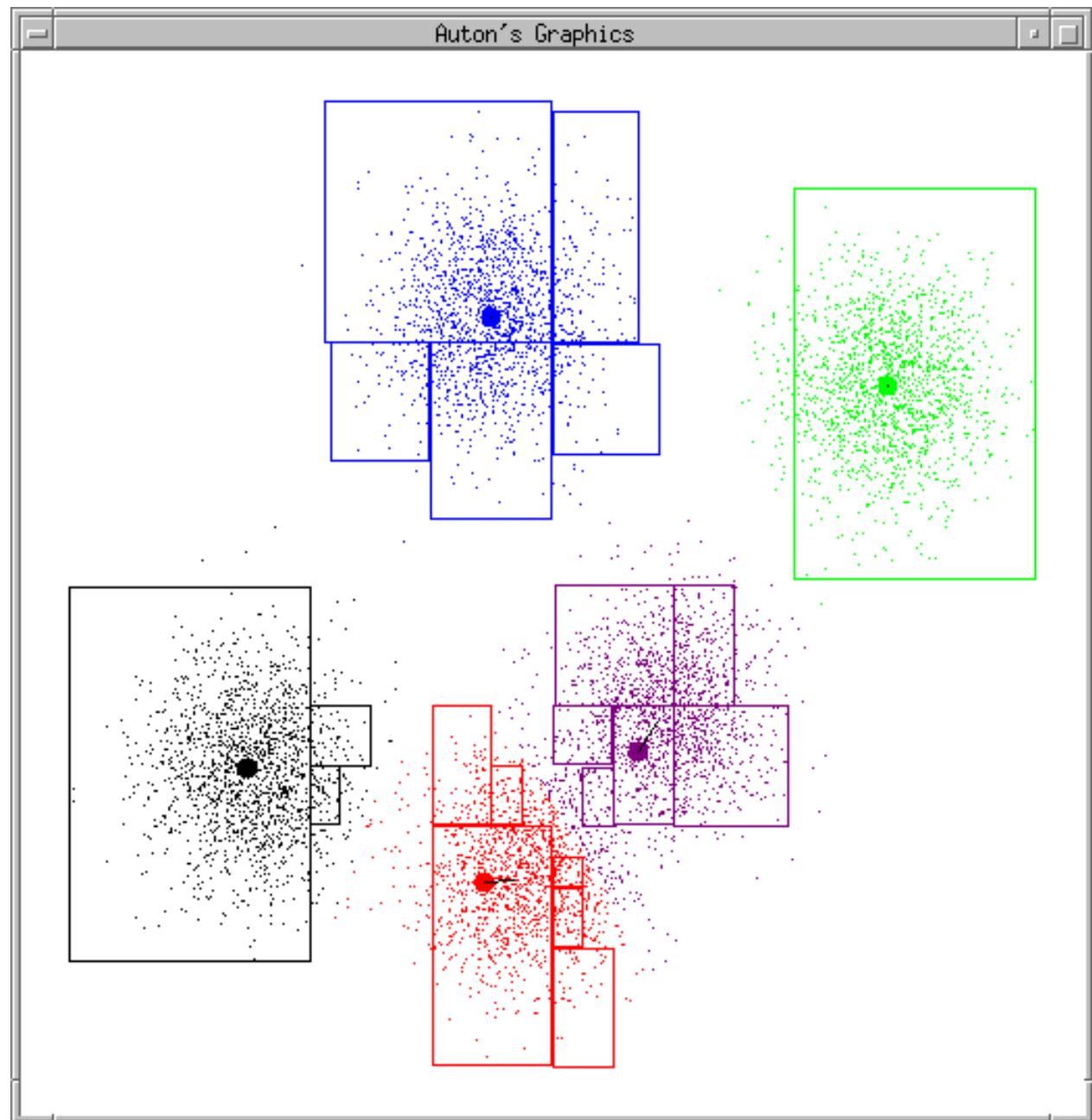
K-means
continues...



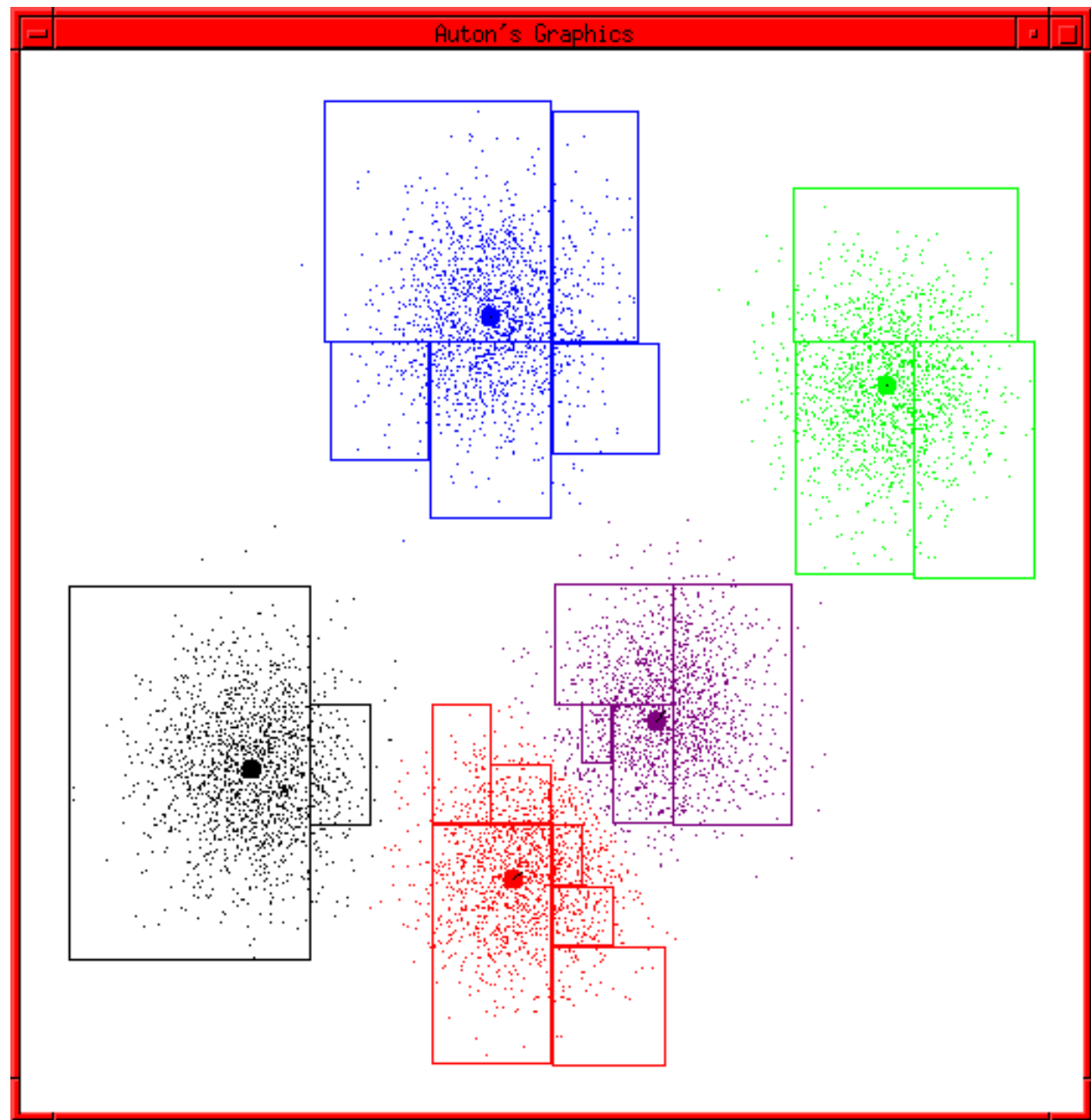
K-means
continues...



K-means
continues...



K-means
terminates



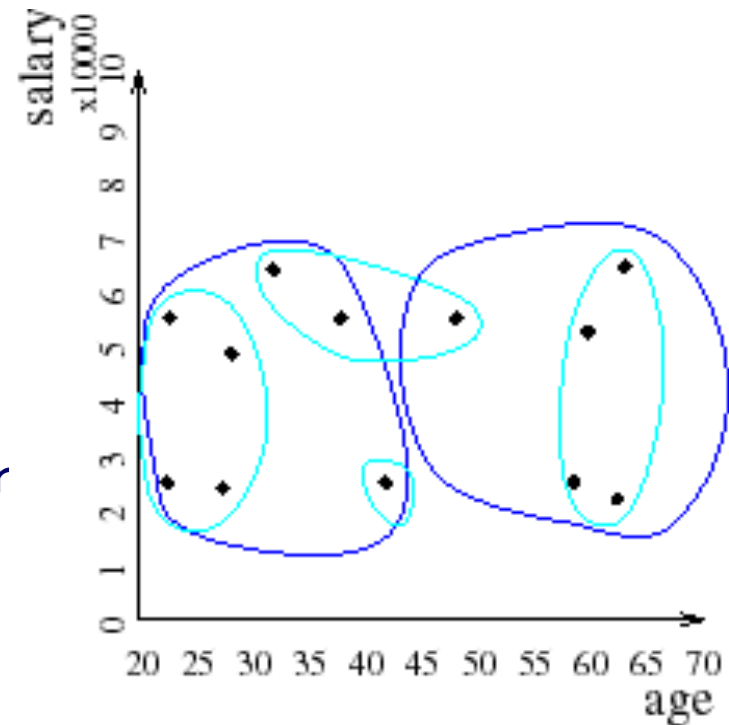
Problems with K-Means type algorithms

■ Advantages

- Relatively efficient: $O(tkn)$,
- where n is the number of objects, k is the number of clusters, and t is the number of iterations.
Normally, $k, t \ll n$.
- Often terminates at a local optimum.

■ Problems

- Clusters are approximately spherical
- Unable to handle noisy data and outlier
- **High dimensionality** may be a problem
- The value of k is an input parameter



Spectral Clustering (I)

- **Algorithms that cluster points using eigenvectors** of matrices derived from the data
- Obtain data representation in the **low-dimensional space** that can be easily clustered
- Variety of methods that use the eigenvectors differently

[Ng, Jordan, Weiss. NIPS 2001]

[Belkin, Niyogi, NIPS 2001]

[Dhillon, KDD 2001]

[Bach, Jordan NIPS 2003]

[Kamvar, Klein, Manning. IJCAI 2003]

[Jin, Ding, Kang, NIPS 2005]

Spectral Clustering methods

- **Method #1**
 - Partition using only one eigenvector at a time
 - Use procedure recursively
 - **Example:** Image Segmentation
- **Method #2**
 - Use k eigenvectors (k chosen by user)
 - Directly compute k -way partitioning
 - Experimentally it has been seen to be “better” ([Ng, Jordan, Weiss. NIPS 2001][Bach, Jordan, NIPS '03]).

Kernel-based k-means clustering

(Dhillon et al., 2004)

- Data not **linearly separable**
- **Transform data to high-dimensional space** using kernel
 - φ a function that maps X to a high dimensional space
- Use the kernel trick to evaluate the dot products:
 - a kernel function $k(x, y)$ computes $\varphi(x) \cdot \varphi(y)$
- cluster kernel similarity matrix using **weighted kernel K-Means**.
- The goal is to minimize the following objective function:

$$J(\{\pi_c\}_{c=1}^k) = \sum_{c=1}^k \sum_{x_i \in \pi_c} \alpha_i \|\varphi(x_i) - m_c\|^2$$

$$\text{where } m_c = \frac{\sum_{x_i \in \pi_c} \alpha_i \varphi(x_i)}{\sum_{x_i \in \pi_c} \alpha_i}$$

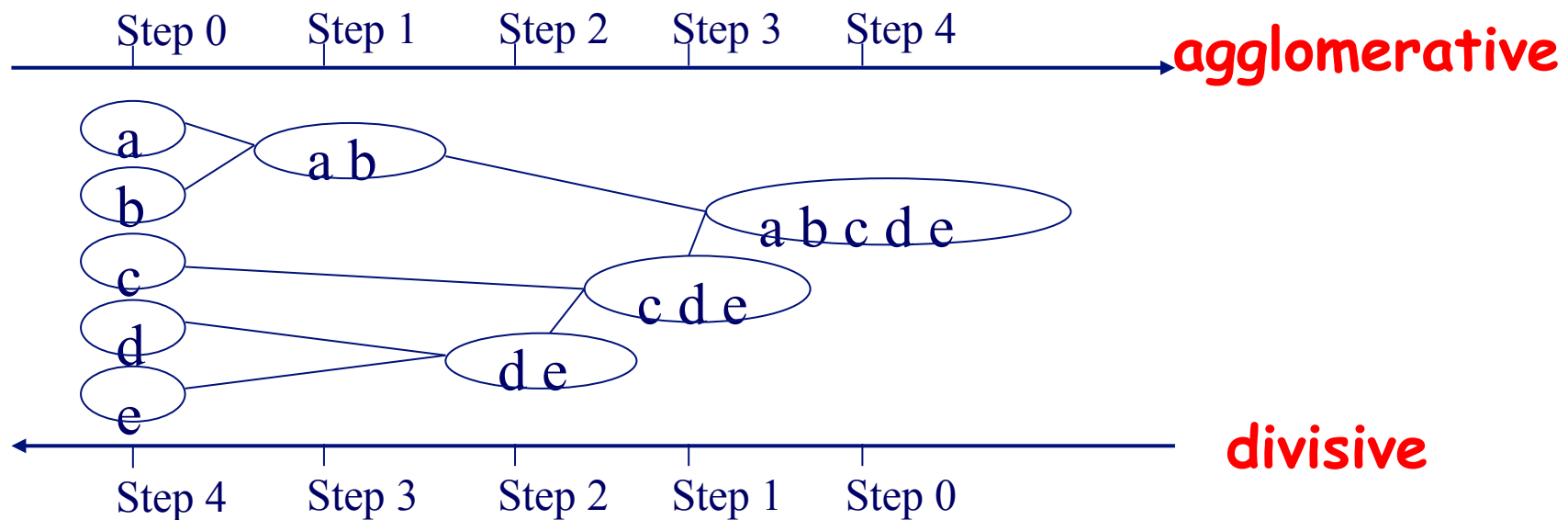
Hierarchical Clustering

- **Two basic approaches:**

- merging smaller clusters into larger ones (**agglomerative**),
- splitting larger clusters (**divisive**)

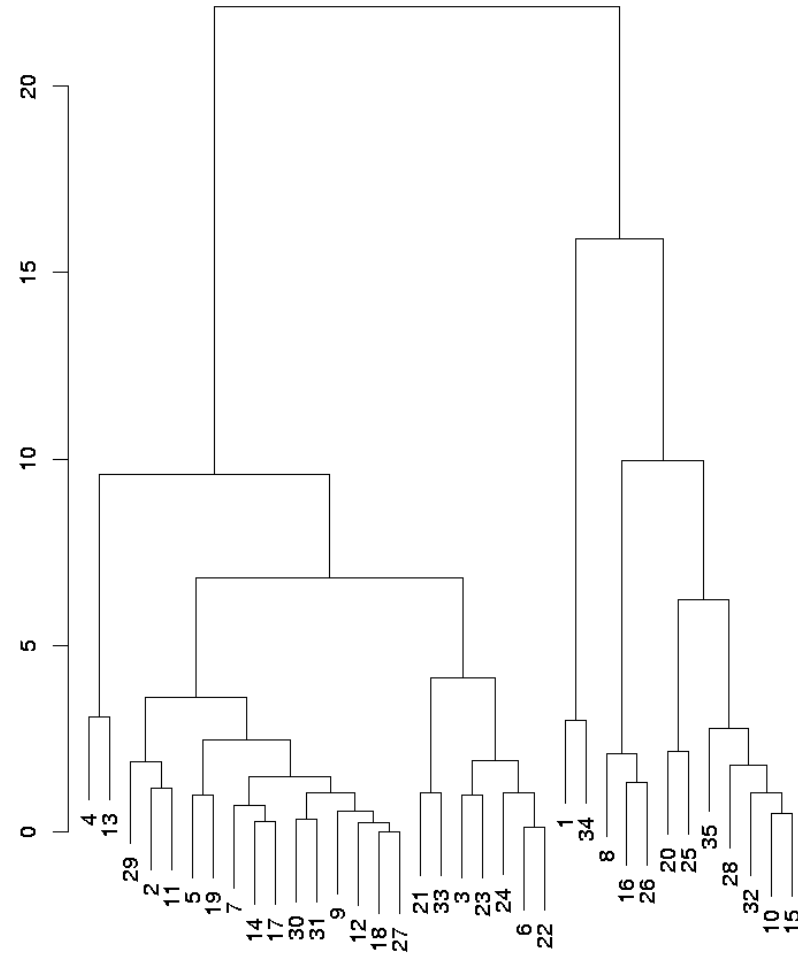
- visualize both via **“dendograms”**

- ✓ shows nesting structure
- ✓ merges or splits = tree nodes

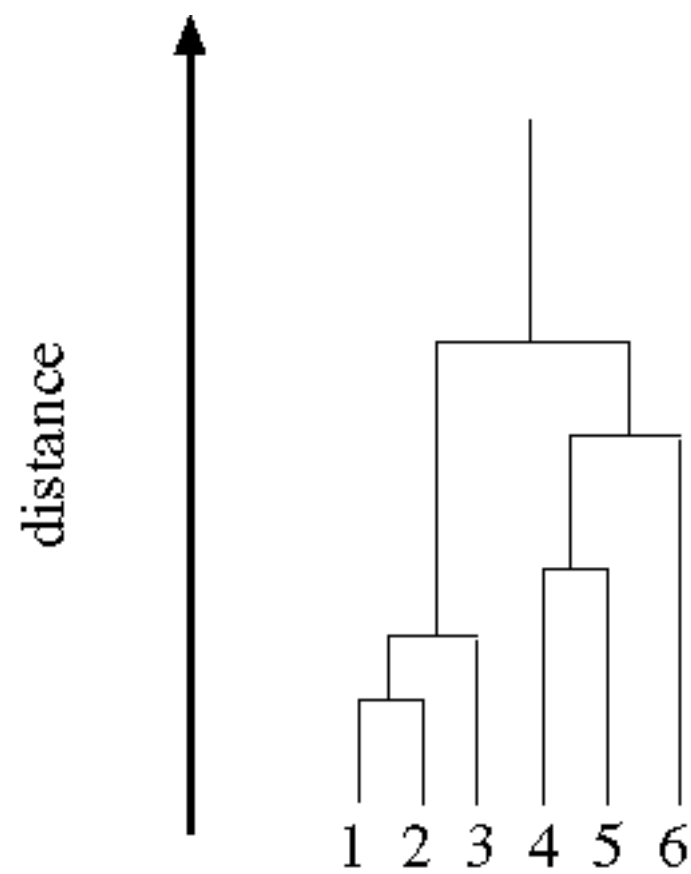
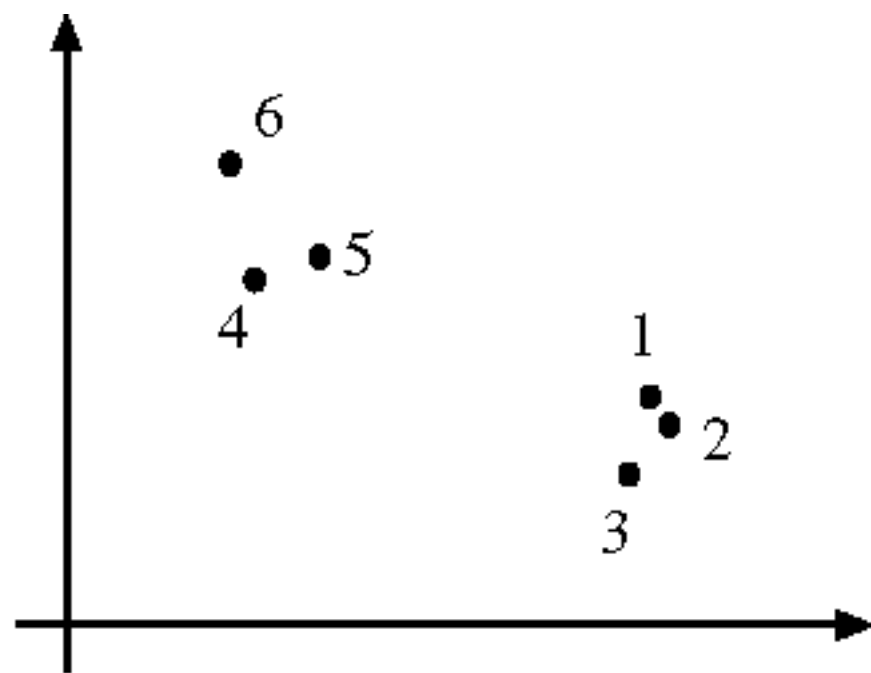


Hierarchical Clustering: Complexity

- Quadratic algorithms
- Running time can be improved using sampling [Guha et al, SIGMOD 1998] or using the triangle inequality (when it holds)



*based on slides by Padhraic Smyth UC, Irvine



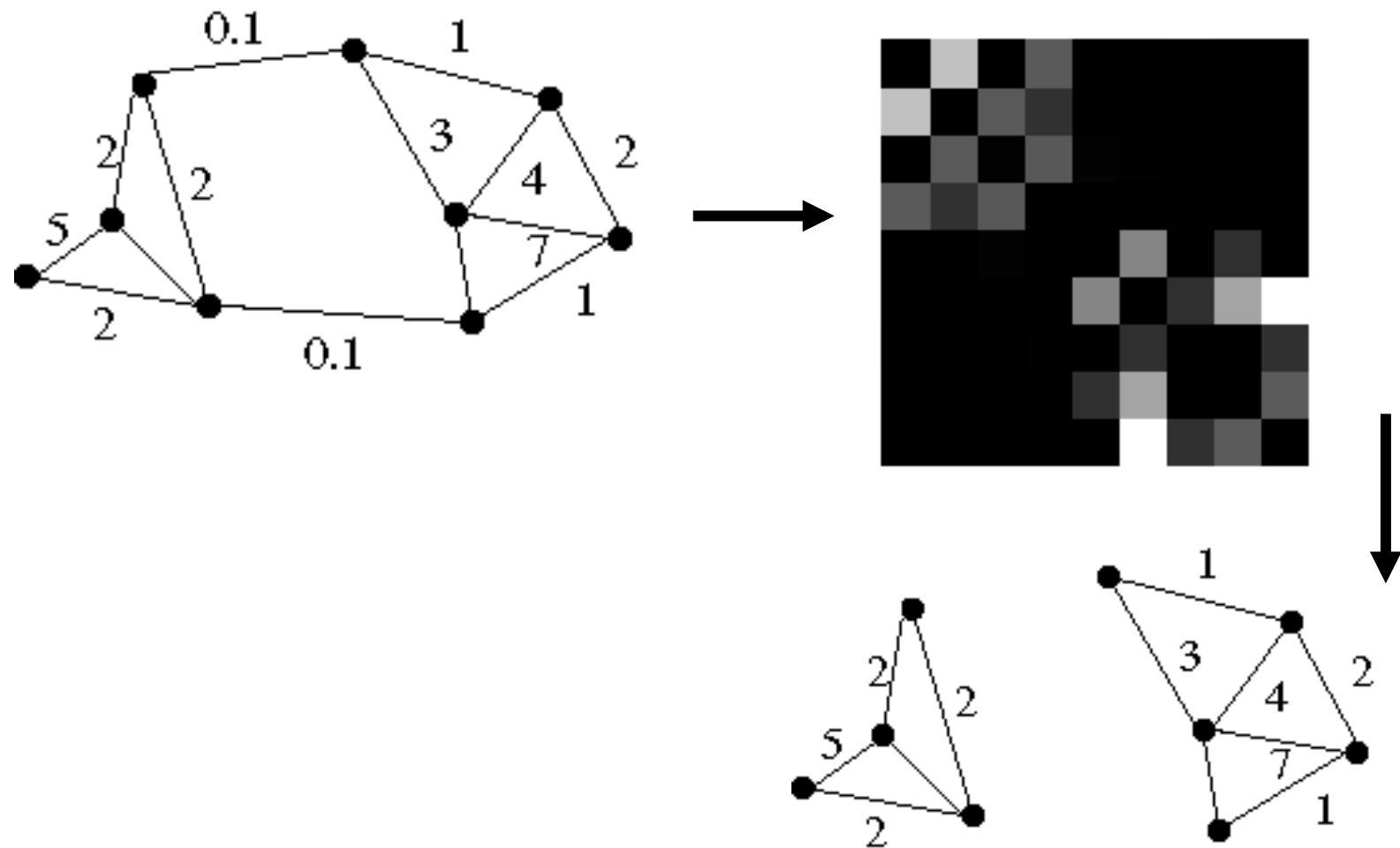
Agglomerative Methods: Bottom-Up

- algorithm based on distance between clusters:
 - for $i=1$ to n let $C_i = \{ x(i) \}$ -- i.e. start with n singletons
 - while more than one cluster left
 - let C_i and C_j be cluster pair with minimum distance over $\text{dist}[C_i, C_j]$
 - merge them, via $C_i = C_i \cup C_j$ and remove C_j
- time complexity = $O(n^2)$ to $O(n^3)$
 - n iterations (start: n clusters; end: 1 cluster)
 - 1st iteration $O(n \lg n)$ to $O(n^2)$ to find nearest singleton pair
- space complexity = $O(n \lg n)$ to $O(n^2)$
 - accesses all/most distances between $x(i)$'s during build
 - interpreting large n dendrogram difficult anyway (like DTs)
- large n idea: partition-based clusters at leafs

Distances Between Clusters

- single link / nearest neighbor measure:
 - $D(C_i, C_j) = \min \{ d(x, y) \mid x \in C_i, y \in C_j \}$
 - can be outlier/noise sensitive
- complete link / furthest neighbor measure:
 - $D(C_i, C_j) = \max \{ d(x, y) \mid x \in C_i, y \in C_j \}$
- intermediates between those extremes:
 - average link: $D(C_i, C_j) = \text{avg} \{ d(x, y) \mid x \in C_i, y \in C_j \}$
 - centroid: $D(C_i, C_j) = d(c_i, c_j)$ where c_i, c_j are centroids
 - Wards's SSE measure (for vector data):
 - within-cluster sum-of-squared-dists for C_i + for C_j - for merged
- DM theme: try several, see which is most interesting

Spectral/Graph-based Clustering

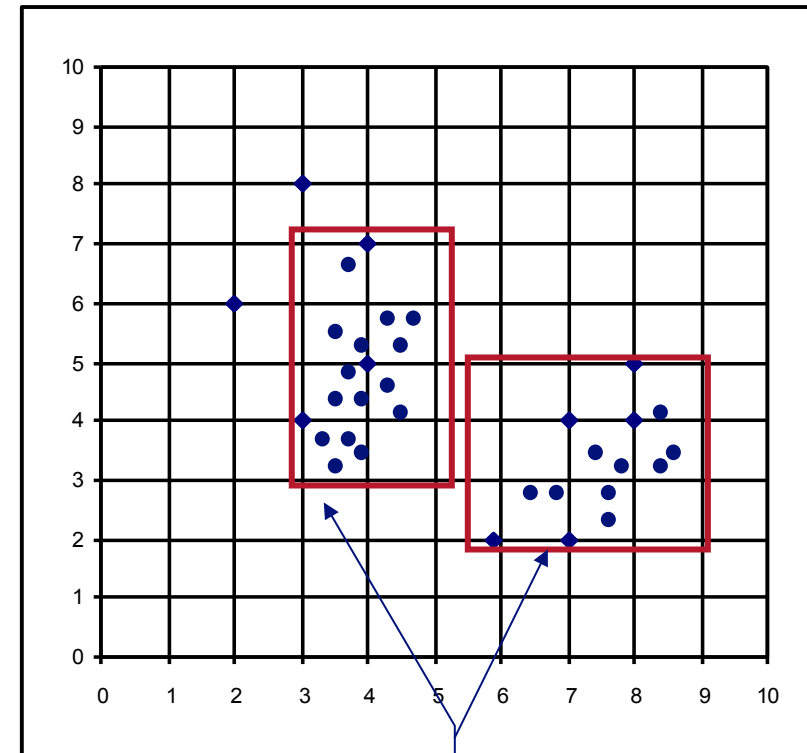


Density-based Clustering Algorithms

- **Clustering** based on density (local cluster criterion), such as density-connected points
- **Major features:**
 - ✓ Discover clusters of arbitrary shape
 - ✓ Handle noise
 - ✓ Need density parameters as termination condition
 - ✓ Work for low dimensional spaces
- **Representative algorithms:**
 - ✓ **DBSCAN:** Ester, et al. (KDD'96)
 - ✓ **DENCLUE:** Hinneburg & D. Keim (KDD'98)

Density-based Algorithms

- Clusters are **regions of space which have a high density of points**
- Clusters can have **arbitrary shapes**

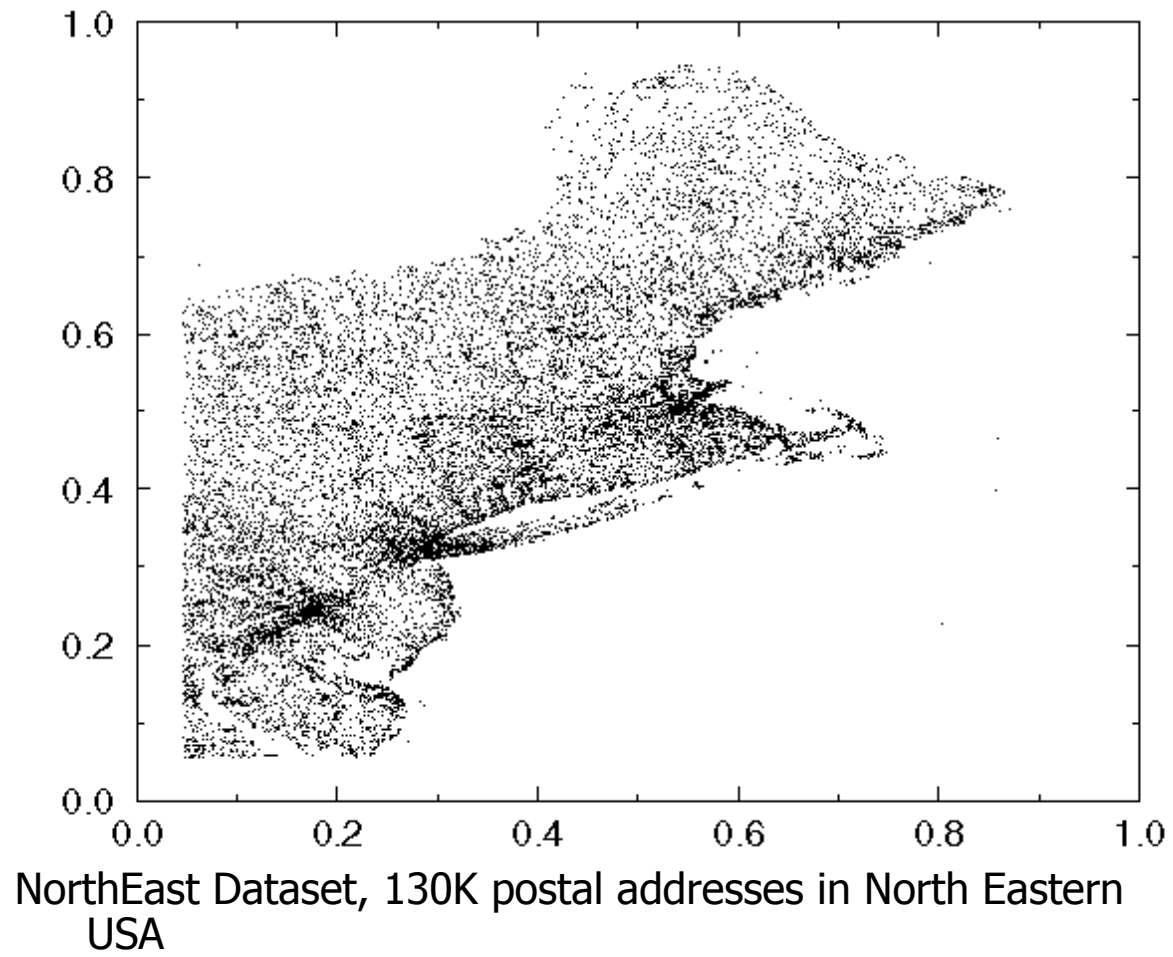


Regions of
high density

Data Reduction: Random Sampling

- **Random Sampling** is used as a data reduction method
- **Idea:** Use a random sample of the dataset and run the clustering algorithm over the sample
- Used extensively for **clustering** [Ng and Han 94, Guha et al 98]
- **But:**
 - For datasets that contain clusters with different densities, we may miss some sparse ones
 - For datasets with noise we may include significant amount of noise in our sample

Example: NorthEast Dataset



Clustering High Dimensional Data

- Fundamental to all clustering techniques is the choice of distance measure between data points;

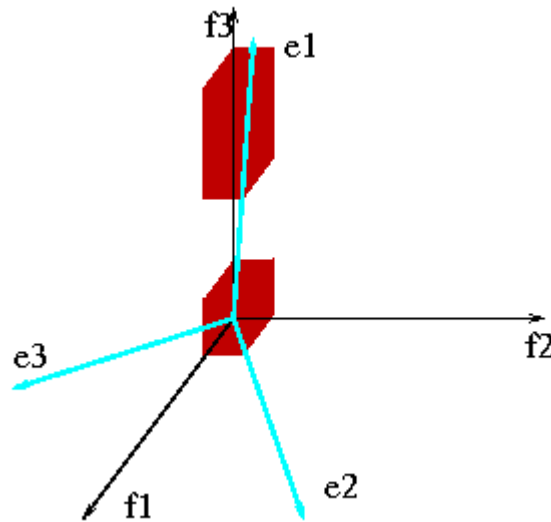
$$D(\mathbf{x}_i, \mathbf{x}_j) = \sum_{k=1}^q (x_{ik} - x_{jk})^2$$

- Assumption:** All features are **equally important**;
- Such approaches fail in high dimensional spaces
- Feature selection (Dy and Brodley, 2000)

Dimensionality Reduction

Applying Dimensionality Reduction Techniques

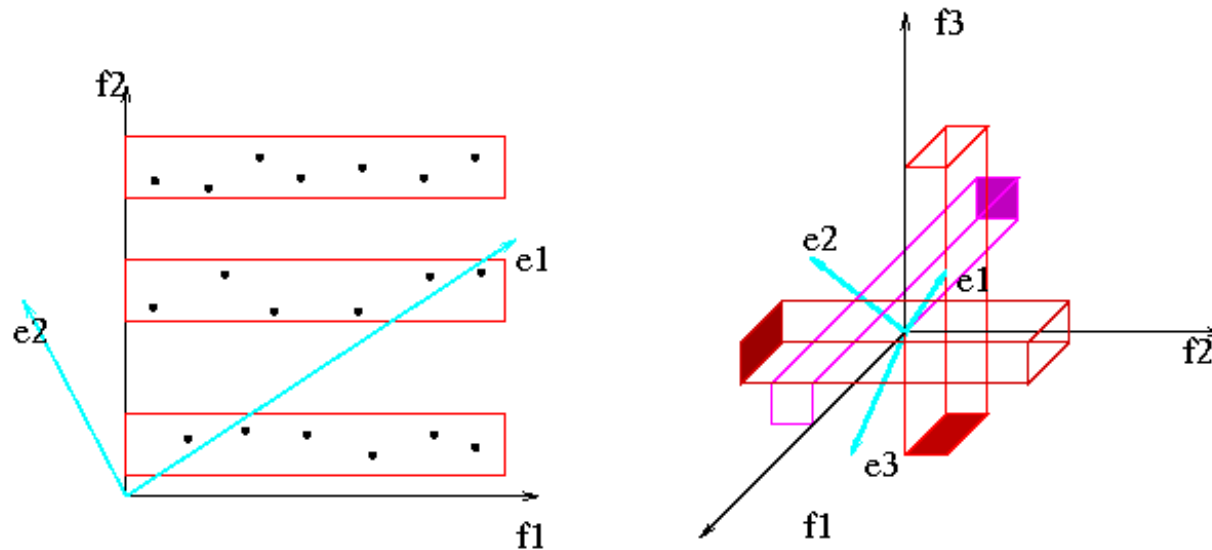
Dimensionality reduction techniques (such as **Singular Value Decomposition**) can provide a solution by reducing the dimensionality of the dataset:



Drawbacks:

- The new dimensions may be difficult to interpret
- They don't improve the clustering in all cases

Applying Dimensionality Reduction Techniques



Different dimensions may be relevant to different clusters

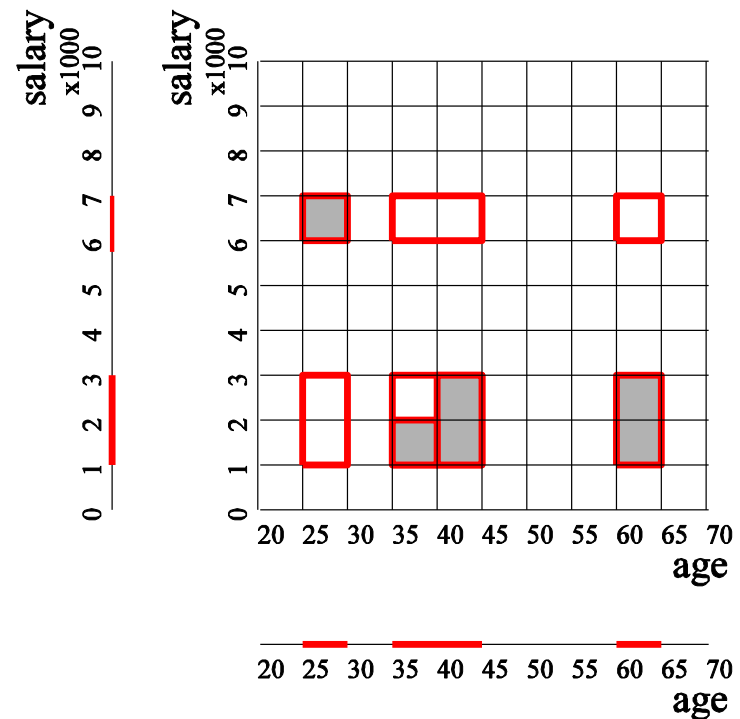
In General: Clusters may exist in different subspaces, comprised of different combinations of features

Subspace clustering

- **Subspace clustering** addresses the problems that arise from high dimensionality of data
 - It finds clusters in subspaces: subsets of the attributes
- Density based techniques
 - **CLIQUE**: Agrawal, Gehrke, Gunopulos, Raghavan (SIGMOD' 98)
 - **DOC**: Procopiuc, Jones, Agarwal, and Murali, (SIGMOD, 2002)
- Iterative algorithms
 - **PROCLUS**: Agrawal, Procopiuc, Wolf, Yu, Park (SIGMOD'99)
 - **ORCLUS**: Aggarwal, and Yu (SIGMOD 2000).

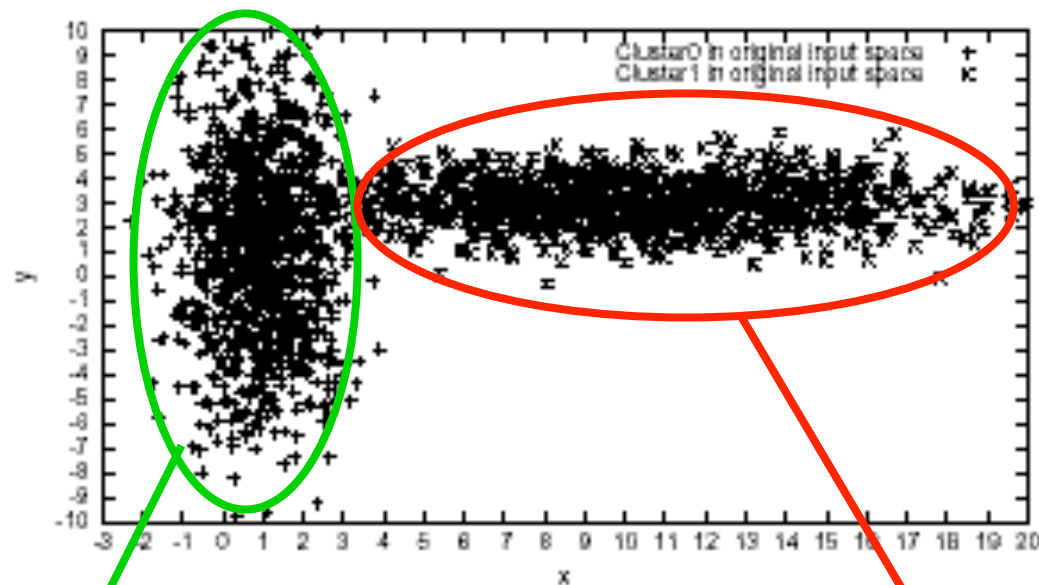
Subspace clustering

- **Density based clusters:** find dense areas in subspaces
- Identifying the right sets of attributes is hard
- Assuming a global threshold allows bottom-up algorithms
- Constrained monotone search in a lattice space



Locally Adaptive Clustering

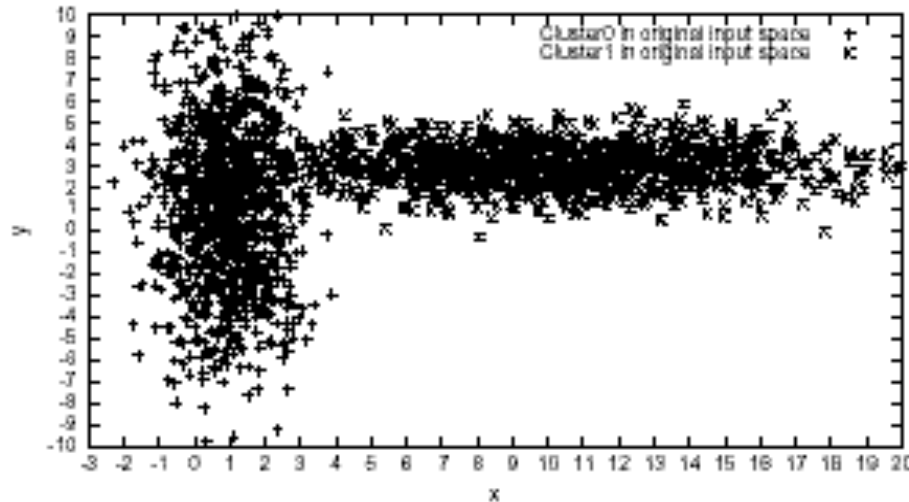
Each cluster is characterized by different attribute weights
(Friedman and Meulman 2002, Domeniconi 2004)



$$(w_{1x}, w_{1y}), w_{1x} > w_{1y}$$

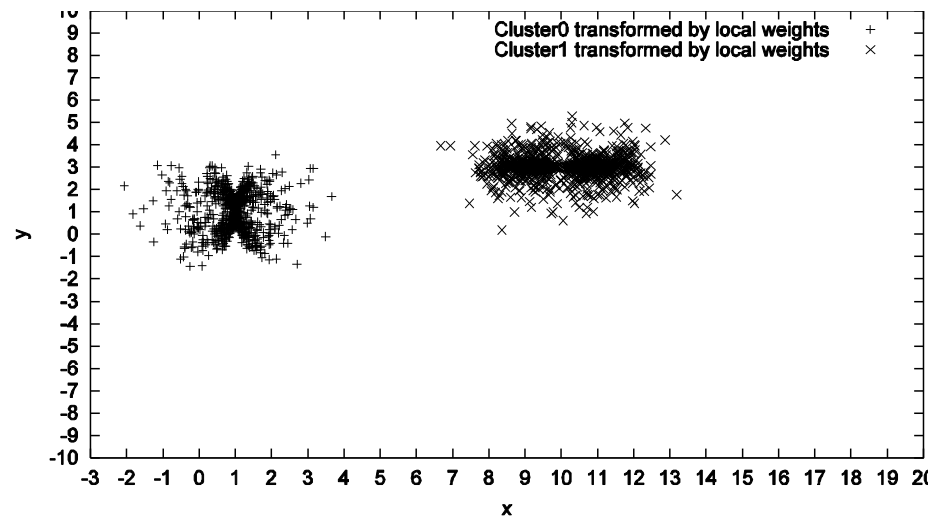
$$(w_{2x}, w_{2y}), w_{2y} > w_{2x}$$

Locally Adaptive Clustering : Example



before local
transformations

after local
transformations



LAC

[C. Domeniconi et al SDM04]

- **Computing the weights:**

X_{ji} : average squared distance along dimension i of points in S_j from c_j

$$X_{ji} = \frac{1}{|S_j|} \sum_{x \in S_j} (c_{ji} - x_i)^2$$

$$w_{ji} = \frac{e^{-X_{ji}}}{\sum_l e^{-X_{jl}}}$$

Exponential weighting scheme

Result :

$w_1, w_2, \dots, w_k$

A weight vector for each cluster

Convergence of LAC

The **LAC algorithm** converges to a local minimum of the error function:

$$E(C, W) = \sum_{j=1}^k \sum_{i=1}^q w_{ji} e^{-X_{ji}}$$

subject to the constraints $\sum_{i=1}^q w_{ji}^2 = 1 \quad \forall j$

$$C = [c_1 \cdots c_k] \quad W = [w_1 \cdots w_k]$$

EM-like convergence:

Hidden variables: assignments of points to centroids (S_j)

E-step: find the values of S_j given w_{ji}, c_{ji}

M-step: find w_{ji}, c_{ji} that minimize $E(C, W)$ given current estimates S_j .

Semi-Supervised Clustering

- **Clustering** is applicable in many real life scenarios
 - there is typically a large amount of **unlabeled data** available.
- The use of **user input** is critical for
 - the success of the clustering process
 - the evaluation of the clustering accuracy.
- **User input** is given as
 - Labeled data
 - Constraints

Learning approaches that use

labeled data/constraints + unlabeled data

have recently attracted the interest of researchers

Motivating semi-supervised learning

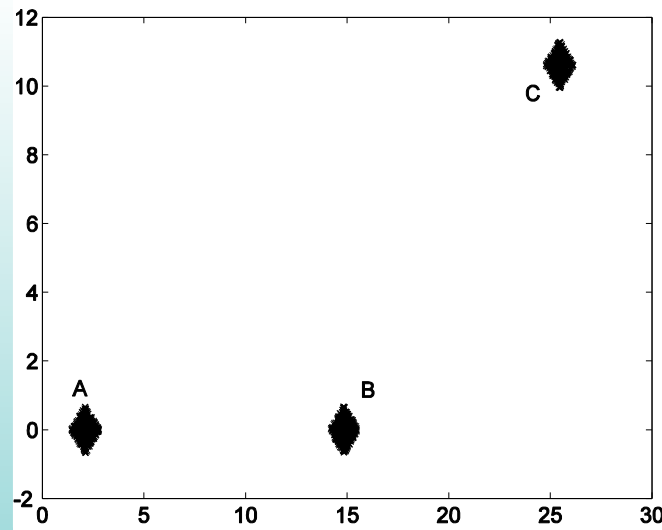
- **Data are correlated.** To recognize clusters, a distance function should reflect such correlations.
- **Different attributes may have different degree of relevance** depending on the application / user requirements
- ☹ A clustering algorithm does not provide the criterion to be used.



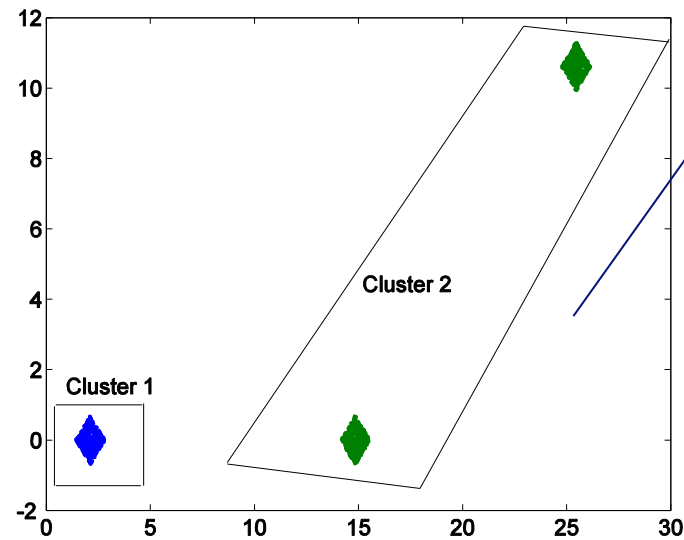
Semi-supervised algorithms: Define clusters taking into account

- ***labeled data or constraints***

if we have “labels” we will convert them to “constraints”

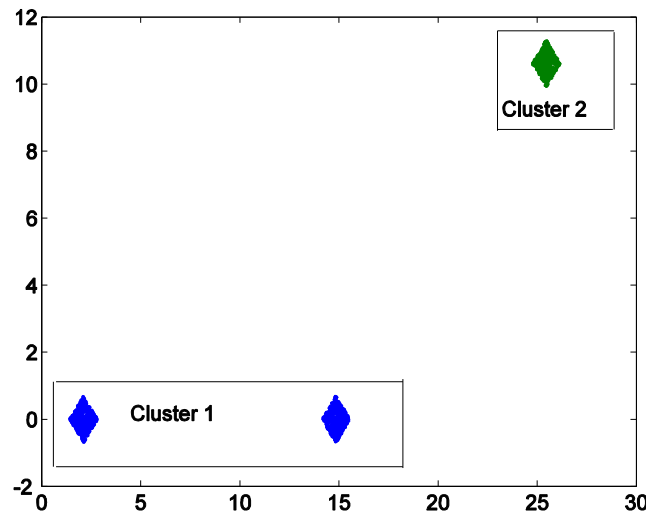


(a)

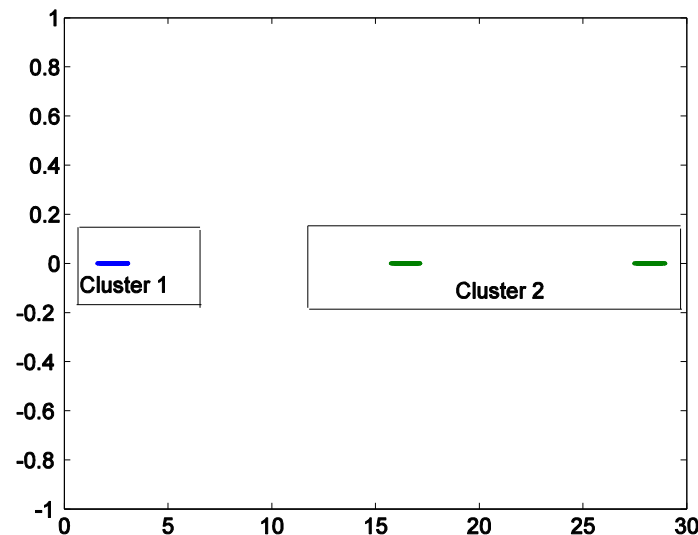


(b)

a user may want the points in B and C to belong to the same cluster



(c)



→ The right clustering may depend on the user's perspective.

→ Fully automatic techniques are very limited in addressing this problem

Clustering under constraints

- Use **constraints** to
 - learn a **distance function**
 - Points surrounding a pair of **must-link/cannot-link** points should be close to/far from each other
 - guide the algorithm to a useful solution
 - Two points should be in the same/different clusters

Defining the constraints

- A set of points $X = \{x_1, \dots, x_n\}$ on which sets of *must-link(S)* and *cannot-link constraints(D)* have been defined.
- **Must-link constraints**
 - $S: \{(x_i, x_j) \text{ in } X\} : x_i \text{ and } x_j \text{ should belong to the same cluster}$
- **Cannot-link constraints**
 - $D: \{(x_i, x_j) \text{ in } X\} : x_i \text{ and } x_j \text{ cannot belong to the same cluster}$
- **Conditional constraints**
 - δ -constraint and ε -constraint

Clustering with constraints: Feasibility issues

- **Constraints** provide information that should be satisfied.
- Options for **constraint-based clustering**
 - Satisfy all constraints
 - Not always possible: A with B, B with C, C not with A.



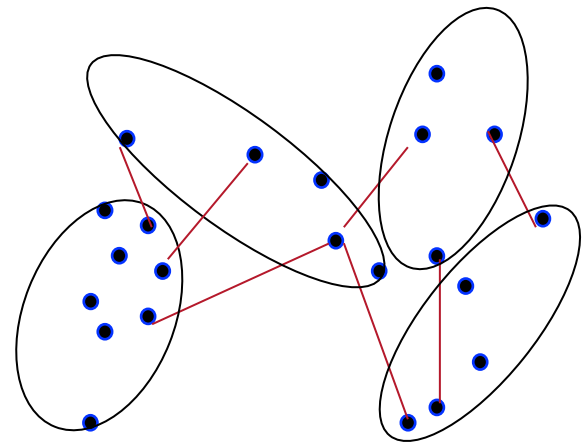
- Satisfy as many constraints as possible

Clustering with constraints: Feasibility issues

- Any combination of constraints involving cannot-link constraints is generally computationally intractable (Davidson & Ravi, ISMB 2000),

- Reduction to k-colorability problem:

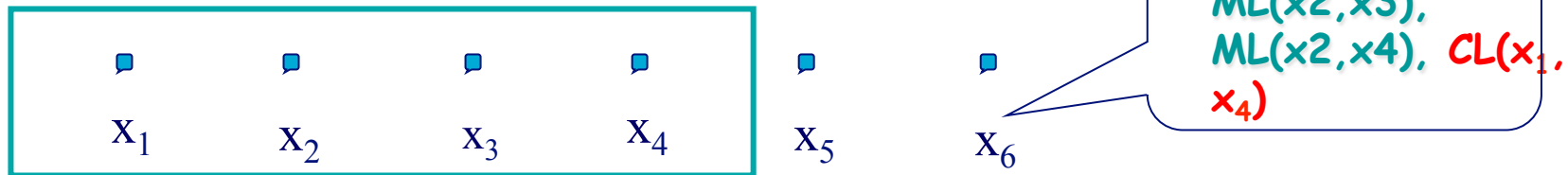
Can you cluster (color) the graph with the cannot-link edges using k colors (clusters)?



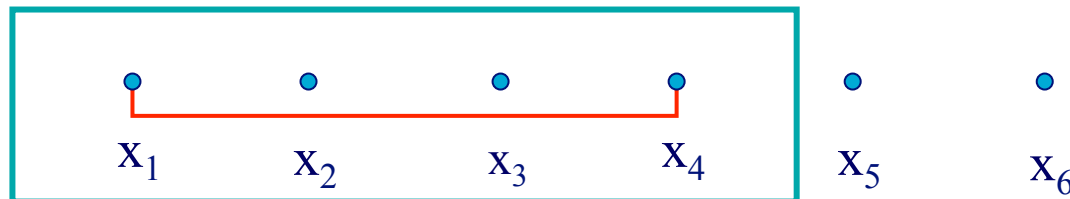
Feasibility under *Must-link(ML)* and *Cannot-link(CL)* constraints



Form the clusters implied by the $ML=\{CC_1 \dots CC_r\}$ constraints $\rightarrow$
Transitive closure of the ML constraints



Construct Edges $\{E\}$ between Nodes based on CL



Infeasible: iff $\exists h, k : e_h(x_i, x_j) : x_i, x_j \in CC_k$

Feasibility under ML and ε

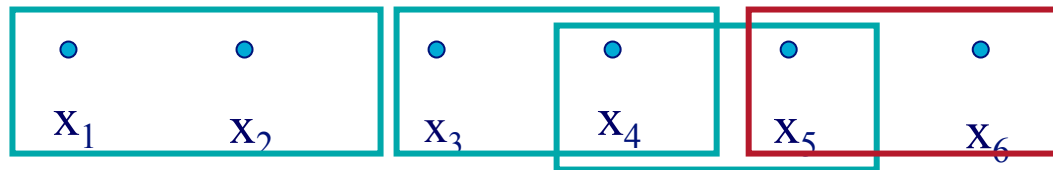
ε -constraint: Any node x should have an ε -neighbor in its cluster (another node y such that $D(x,y) \leq \varepsilon$)

$S' = \{x \in S : x \text{ does not have an } \varepsilon \text{ neighbor}\} = \{s_5, s_6\}$

Each of these should be in their own cluster



Compute the *Transitive Closure* on $ML = \{CC_1 \dots CC_r\}$



$ML(x_1, x_2),$
 $ML(x_3, x_4),$
 $ML(x_4, x_5)$

Infeasible: iff $\exists i, j : x_i \in CC_j, x_i \in S'$

Clustering based on constraints

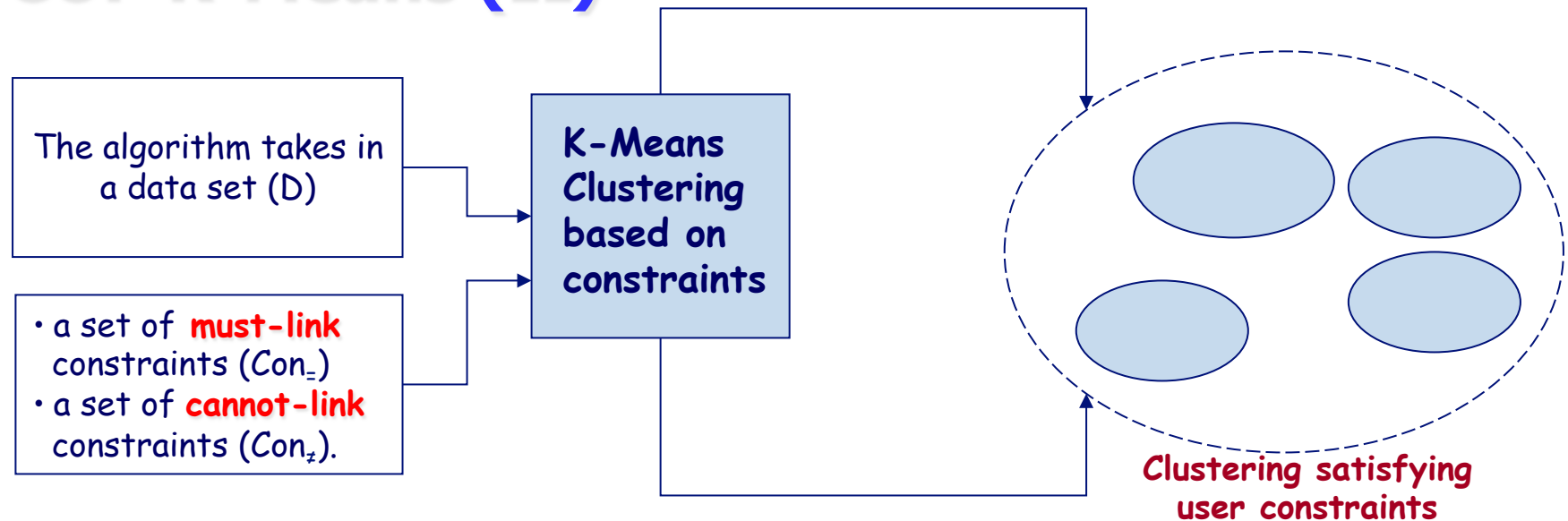
- **Algorithm specific approaches**
 - Incorporate constraints into the clustering algorithm
 - COP K-Means (Wagstaff et al, 2001)
 - Hierarchical clustering (I. Davidson, S. Ravi, 2005)
 - Incorporate metric learning into the algorithm
 - MPCK-Means (Bilenko et al 2004)
 - HMRF K-Means (Basu et al 2004)
- **Learning a distance metric** (Xing et al. '02)
- **Kernel-based constrained clustering** (Kulis et al.'05)

COP K-Means (I)

[Wagstaff et al, 2001]

- Semi-supervised variants of K-Means
- **Constraints:** Initial background knowledge
- **Must-link & Cannot-link** constraints are used in the clustering process
 - Generate a partition that satisfies all the given constraints

COP K-Means (II)



- **When updating cluster assignments,**
 - we ensure that none of the specified constraints are violated.
- **Assign each point d_i to its closest cluster C_j .** This will succeed unless a constraint would be violated.
 - If there is another point $d_{\neq}$ that must be assigned to the same cluster as d , but that is already in some other cluster, or
 - there is another point $d_{\neq}$ that cannot be grouped with d but is already in C , then d cannot be placed in C .
- **Constraints are never broken;** if a legal cluster cannot be found for d , the empty partition (f_g) is returned.

Hierarchical Clustering based on constraints

[I. Davidson, S. Ravi, 2005]

Instance: A set S of nodes, the (symmetric) distance $d(x, y) \geq 0$ for each pair of nodes x and y and a collection C of constraints



- **Question:** Can we create a dendrogram for S so that all the constraints in C are satisfied?

Constraints and Irreducible Clusterings

- A **feasible clustering** $C=\{C_1, C_2, \dots, C_k\}$ of a set S is irreducible if no pair of clusters in C can be merged to obtain a feasible clustering with $k-1$ clusters.

If mergers are not done correctly, the dendrogram may stop prematurely

- $X=\{x_1, x_2, \dots, x_k\},$
 $Y=\{y_1, y_2, \dots, y_k\},$
 $Z=\{z_1, z_2, \dots, z_k\},$
 $W=\{w_1, w_2, \dots, w_k\}$

- **CL-constraints**

- $\forall \{x_i, x_j\}, i \neq j$
- $\forall \{w_i, w_j\}, i \neq j$
- $\forall \{y_i, z_j\}, i \leq j, j \leq i$

- Feasible clustering with $2k$ clusters:
 $\{x_1, y_1\}, \{x_2, y_2\}, \dots, \{x_k, y_k\}, \{z_1, w_1\},$
 $\{z_2, w_2\}, \dots, \{z_k, w_k\}$

But then get stuck

- **Alternative is:**

- $\{x_1, w_1, y_1, y_2, \dots, y_k\}, \{x_2, w_2, z_1, z_2, \dots, z_k\},$
 $\{x_3, w_3\}, \dots, \{x_k, w_k\}$

Learning a distance metric based on user constraints

- The requirement is :
 - learn the distance measure to satisfy user constraints.
- To simplify the problem consider the weighted Euclidean distance:
 - different weights are assigned to different dimensions
 - Other formulations that map the points to a new space can be considered, but are significantly more complex to optimize

Distance Learning as Convex Optimization

[Xing et al. '02]

- **Goal:** Learn a distance metric between the points in X that satisfies the given constraints
- The problem reduces to the following **optimization problem** :

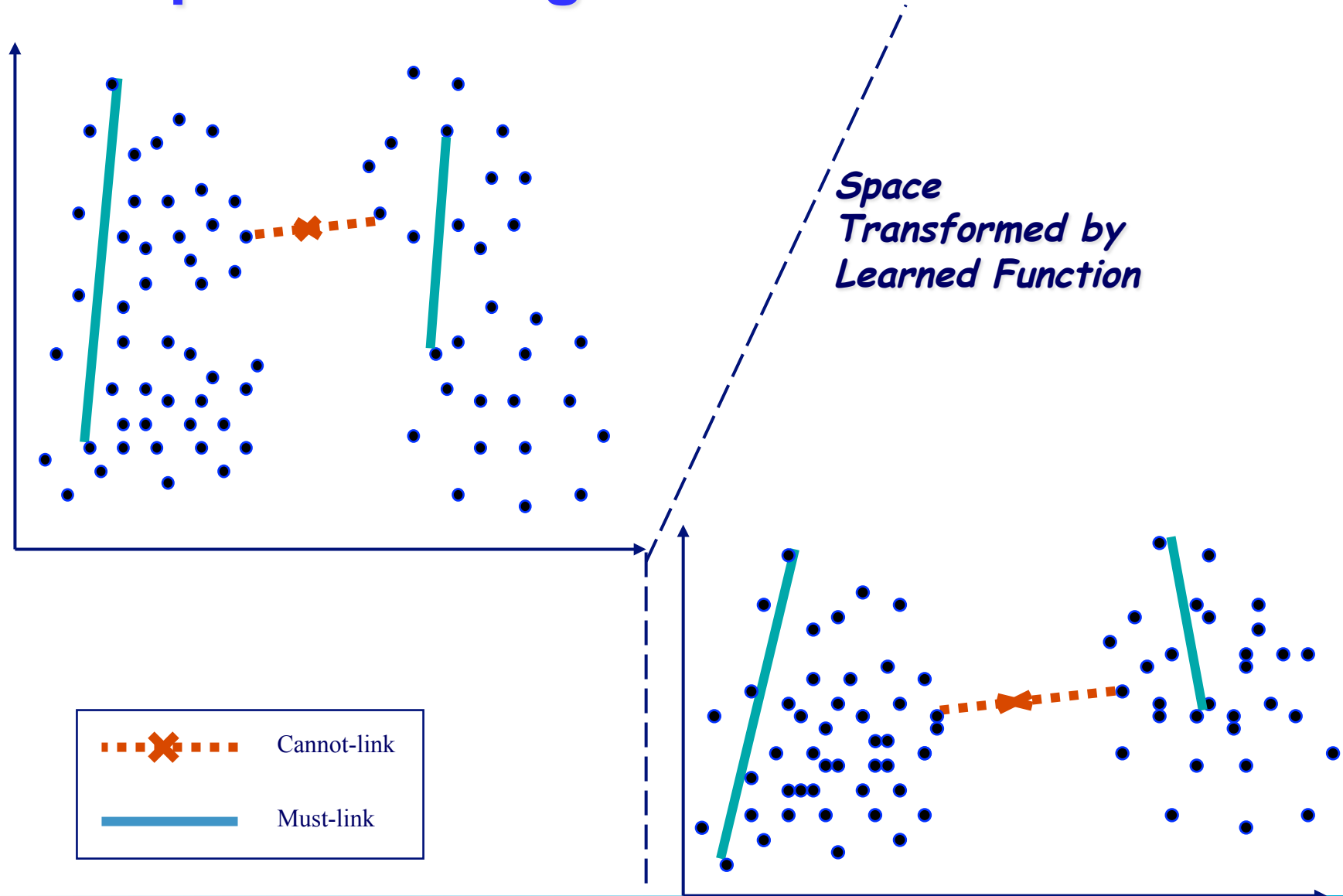
$$\min_A \sum_{(x_i, x_j) \in \text{EML}} \|x_i - x_j\|_A^2$$

given that

$$\sum_{(x_i, x_j) \in \text{ECL}} \|x_i - x_j\|_A \geq 1 \quad A \geq 0$$

E. P. Xing, A. Y. Ng, M. I. Jordan, and S. Russell. Distance metric learning, with application to clustering with side-information. In *NIPS*, December 2002.

Example: Learning Distance Function



Learning Mahalanobis distance

Mahalanobis distance =

Euclidean distance parameterized by matrix A

$$||x - y||_A^2 = (x - y)^T A (x - y)$$

Typically A is diagonal

The Diagonal A Case

- Considering the case of learning *a diagonal A*
- we can solve the original *optimization problem* using Newton-Raphson to efficiently optimize the following

$$g(A) = \sum_{(x_i, x_j) \in \text{EML}} \|x_i - x_j\|_A^2 - \log \left(\sum_{(x_i, x_j) \in \text{ECL}} \|x_i - x_j\|_A \right)$$

Use **Newton Raphson Technique**:

$$x' = x - g(x)/g'(x)$$

$$A' = A - g(A).J^{-1}(A)$$

$$A \geq 0$$

Kernel based Semi-supervised clustering

[Kulis et al.' 05]

A non-linear transformation, ϕ

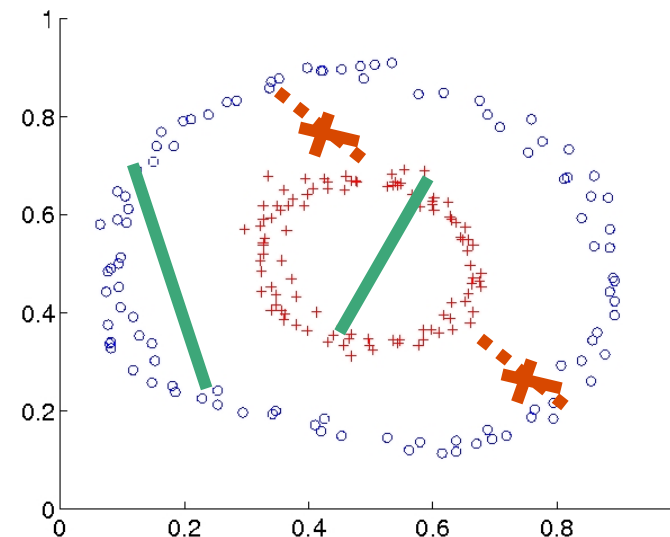
- maps data to a high dimensional space
- the data are expected to be more separable
- a kernel function $k(x, y)$ computes $\phi(x) \cdot \phi(y)$

The user gives constraints

The appropriate kernel is created based on constraints

$$J(\{\pi\}_{c=1}^k) = \sum_{c=1}^k \sum_{x_i \in \pi_c} \|\phi(x_i) - m_c\|^2 - \sum_{\substack{x_i, x_j \in \text{ML} \\ l_i = l_j}} w_{ij} + \sum_{\substack{x_i, x_j \in \text{CL} \\ l_i \neq l_j}} w_{ij}$$

Reward for
constraint
satisfaction



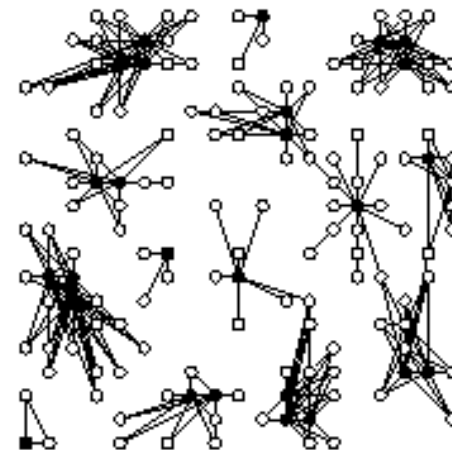
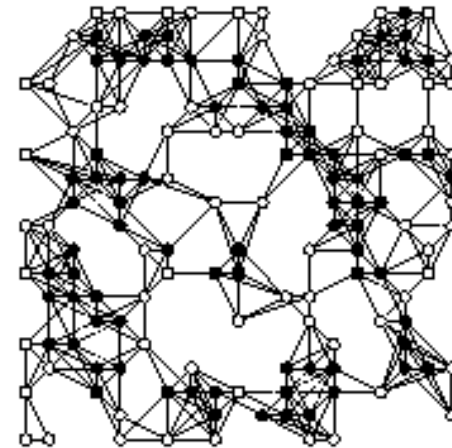
Semi-Supervised Kernel-KMeans

[Kulis et al.' 05]

- **Algorithm:**
 - Constructs the appropriate kernel matrix from data and constraints
 - Runs weighted kernel K-Means
- **Input of the algorithm:** Kernel matrix
 - Kernel function on vector data or
 - Graph affinity matrix
- **Benefits:**
 - HMRF-KMeans and Spectral Clustering are special cases
 - Fast algorithm for constrained graph-based clustering
 - Kernels allow constrained clustering with non-linear cluster boundaries

Graph-based constrained clustering

- **Constrained graph clustering:**
 - minimize cut in input graph while maximally respecting a given set of constraints



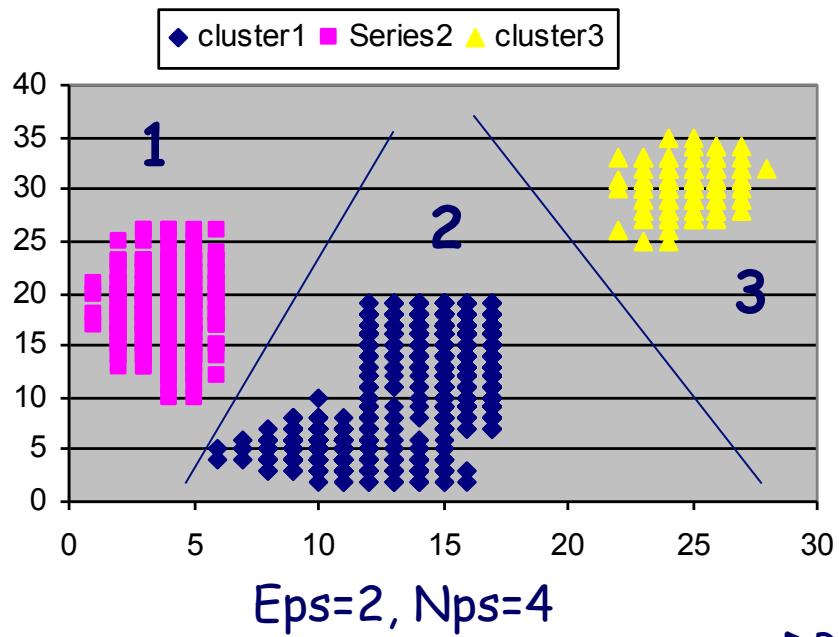
Clustering using constraints and cluster validity criteria

- Different distance metrics may satisfy the same number of constraints
- One solution is to apply a different criterion that evaluates the resulting clustering to choose the right distance metric
- A general approach should:
 - Learn an appropriate distance metric to satisfy the constraints
 - Determine the best clustering w.r.t the defined distance metric.

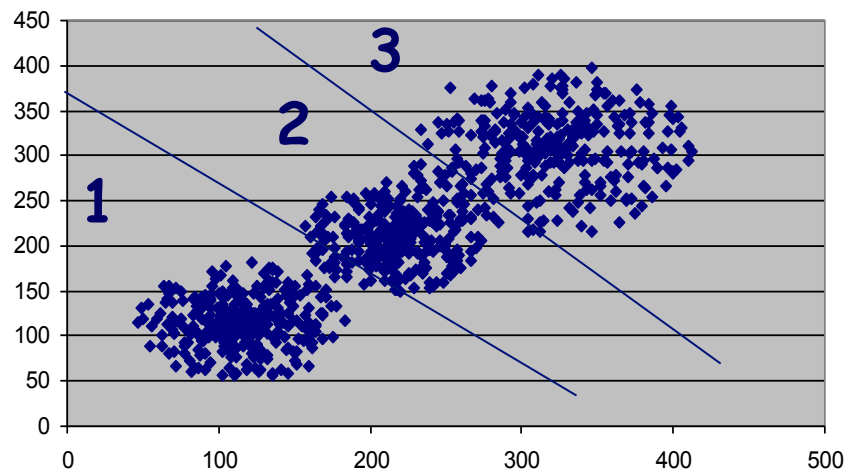
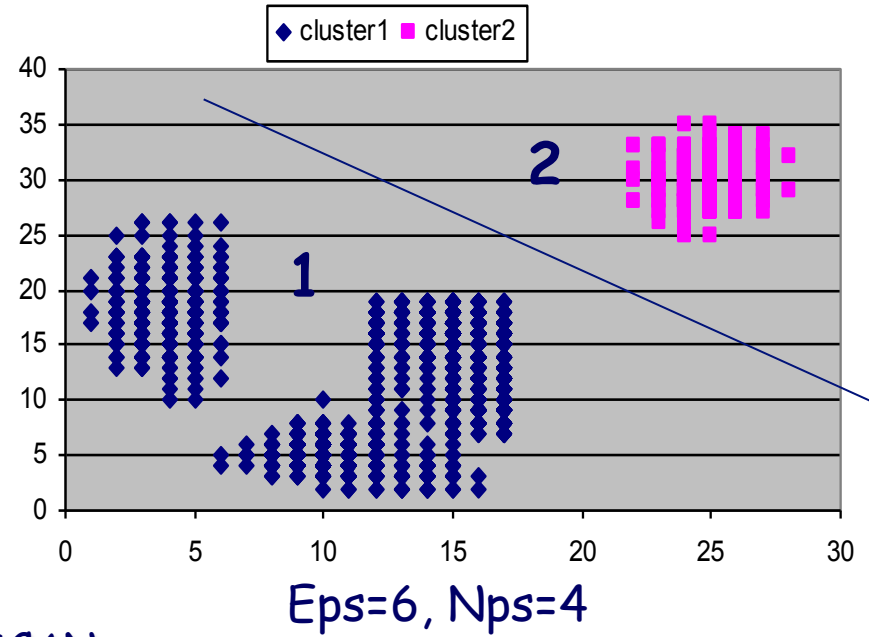
Cluster Validity

A problem we face in clustering is to

- ✓ define the **"best" partitioning** of a data set, i.e.
 - ✓ number of clusters that fits a data set,
 - ✓ capture the shape of clusters presenting in underlying data set
- The clustering results depend on
 - **the data set** (data distribution)
 - Initial **clustering assumptions**, algorithm **input parameters values**

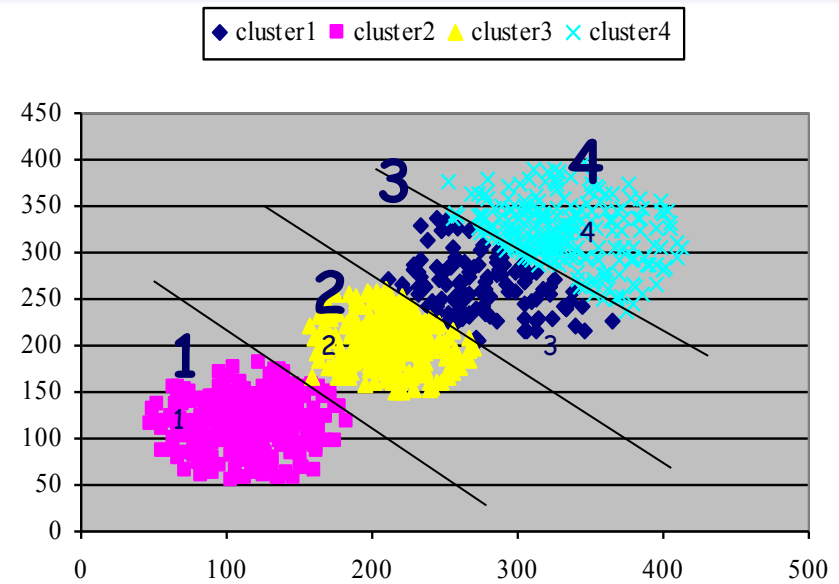


DBSCAN



(a)

K-Means



(b)

S_Dbw cluster validity index [Halkidi, Vazirgiannis, ICDM'01]

- **SDbw**: a relative algorithm-independent validity index, based on
 - Scattering and Density between clusters

Main features of the proposed approach

Validity index **S_Dbw**. Based on the features of the clusters:

- ✓ evaluates the resulting clustering as defined by the algorithm under consideration.
- ✓ selects for each algorithm the optimal set of input parameters with regards to the specific data set.

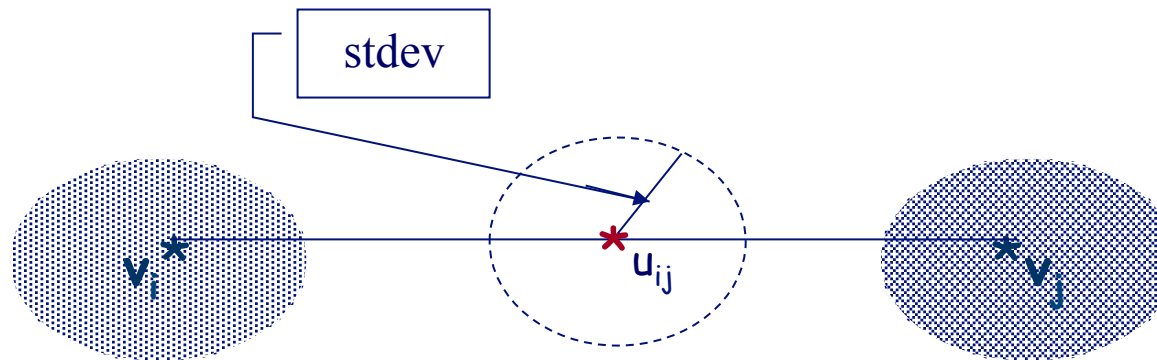
S_Dbw definition: Inter-cluster Density (ID)

Dens_bw: Average density in the area among clusters in relation with the density of the clusters

$$Dens_bw(c) = \frac{1}{c \cdot (c - 1)} \sum_{i=1}^c \left(\sum_{\substack{j=1 \\ i \neq j}}^c \frac{density(u_{ij})}{\max \{density(v_i), density(v_j)\}} \right),$$

$$density(u) = \sum_{l=1}^{n_{ij}} f(x_l, u), f(x, u) = \begin{cases} 0, & \text{if } d(x, u) > stdev \\ 1, & \text{otherwise} \end{cases}$$

where n_{ij} = number of tuples that belong to the clusters c_i and c_j , i.e., $x_l \in c_i \cup c_j \subseteq S$



S_Dbw definition: Intra-cluster variance

Average scattering of clusters:

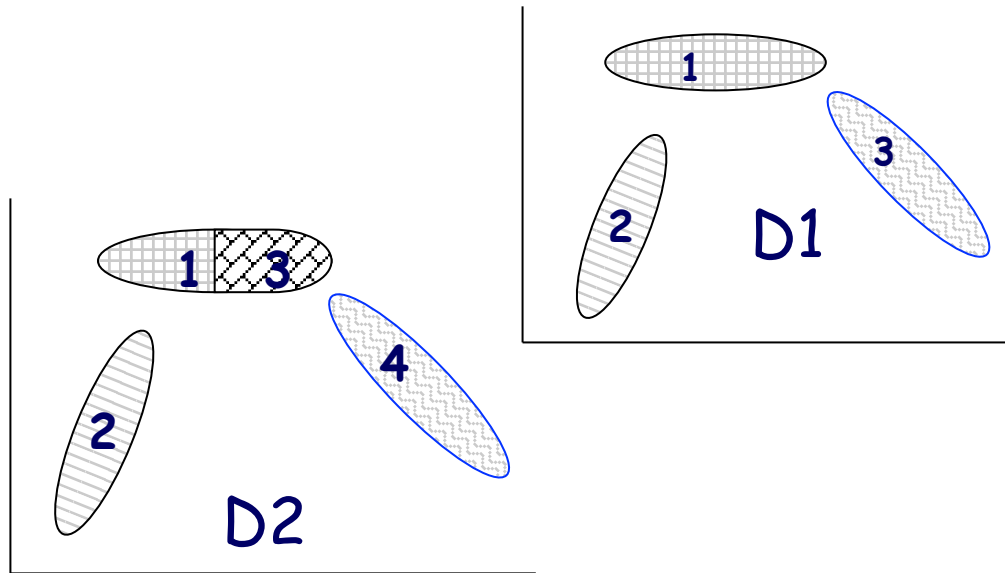
$$Scat(c) = \frac{\frac{1}{c} \sum_{i=1}^c \|\sigma(v_i)\|}{\|\sigma(X)\|}$$

where $\sigma_x^p = \frac{1}{n} \sum_{k=1}^n \left(x_k^p - \bar{x}^p \right)^2$

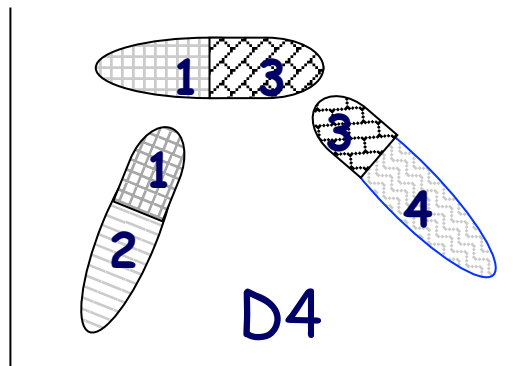
where $\bar{x}^p$ is the p_{th} dimension of $\bar{X} = \frac{1}{n} \sum_{k=1}^n x_k, \forall x_k \in X$

$$\sigma_{v_i}^p = \sum_{k=1}^{n_i} \left(x_k^p - v_i^p \right)^2 / n_i$$

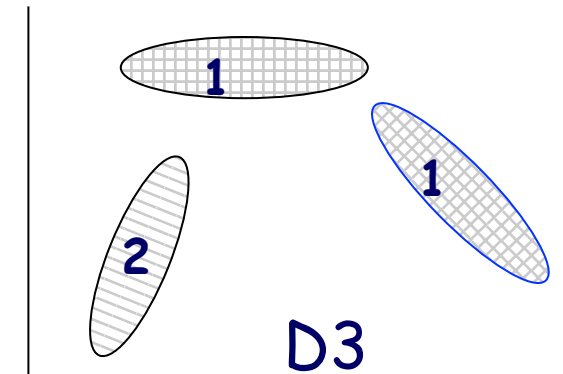
$$\underline{S_Dbw(c) = Scat(c) + Dens_bw(c)}$$



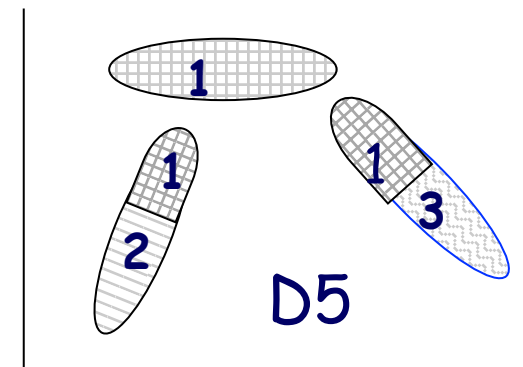
Scat ~ $\searrow$ & Dens_bw $\uparrow$



Scat $\uparrow$ & Dens_bw $\uparrow$



Scat $\uparrow$ & Dens_bw ~



Scat $\uparrow$ & Dens_bw $\uparrow$

Initializing dimension weights based on user constraints

- Learn the distance measure to satisfy user constraints (must-link and cannot-link).
- Different weights are assigned to different dimensions
- Learn *a diagonal* matrix A using Newton-Raphson to efficiently optimize the following equation [Xing et al, 2002]

$$g(A) = \sum_{(x_i, x_j) \in S} \|x_i - x_j\|_A^2 - \log \left(\sum_{(x_i, x_j) \in D} \|x_i - x_j\|_A \right)$$

Best weighting of data dimensions

- **W**: set of different weightings defined for a set of d data dimensions.
- **$W_j \in W$** best weighting for a given dataset
 - **if** the clustering of data in the d -dimensional space defined by

$$W_j = [w_{j1}, \dots, w_{jd}] (w_{ji} > 0)$$

optimizes the quality measure:

$$QoC_{constr}(C_j) = \text{optim}_{i=1, \dots, m} \{QoC_{constr}(C_i)\}$$

given that C_j is the clustering for the W_j weighting vector.

Defining dimension weights

- **Clustering quality criterion (measure)** : evaluates a clustering, C_i , of a dataset in terms of
 - its accuracy w.r.t. the user constraints (ML & CL)
 - its validity based on well-defined cluster validity criteria.

$$QoC_{constr}(C_i) = w \cdot Accuracy_{ML\&CL}(C_i) + ClusterValidity(C_i)$$

significance of the
user constraints
w.r.t. the cluster
validity criteria

% of constraints
satisfied in C_j

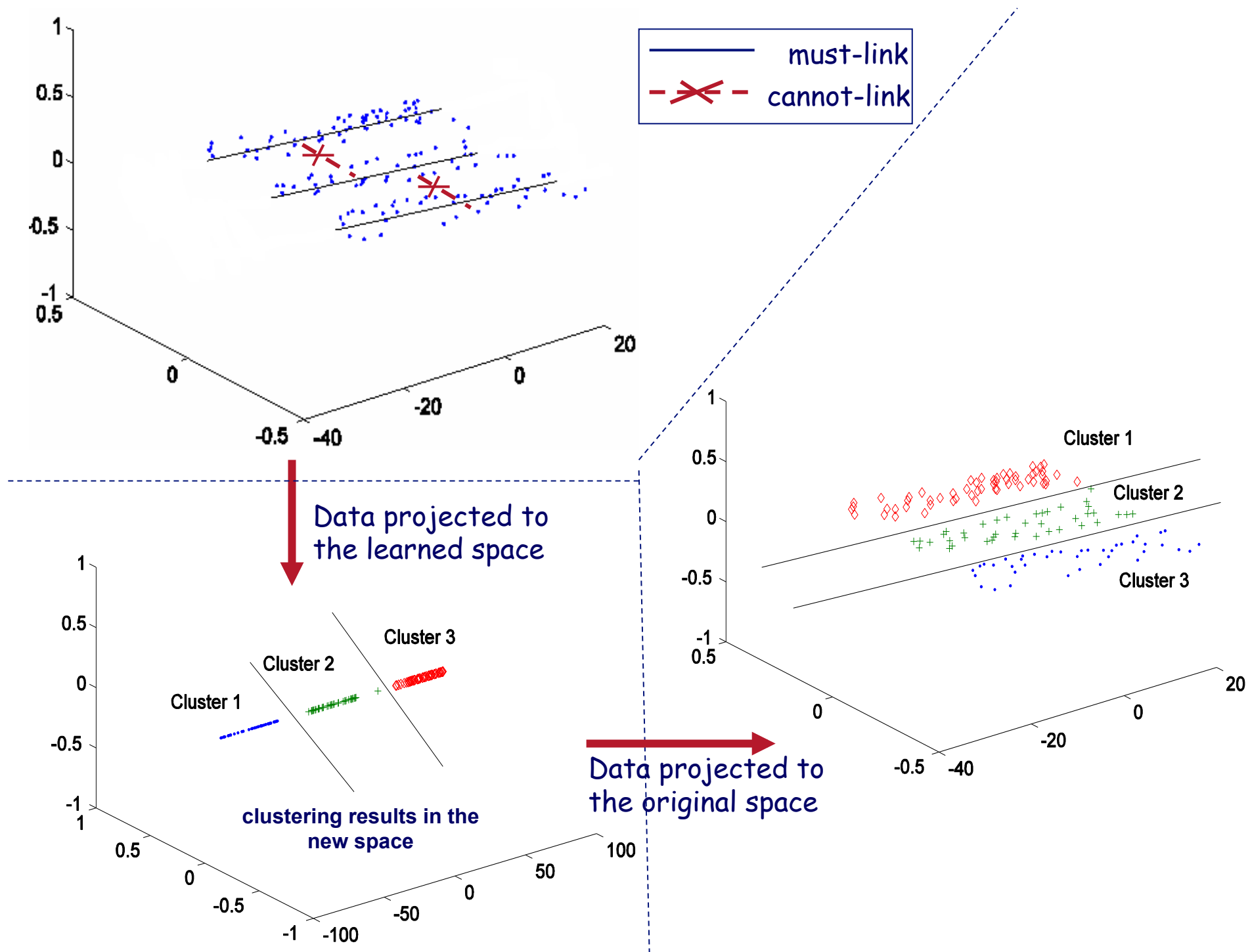
C_i 's cluster
validity.

Hill climbing procedure: Defining dimension weights

- Initialize dimension weights to satisfy ML and CL ,

$$W_{cur} = \{W_i \mid i = 1, \dots, d\}$$

- $Cl_{cur} \leftarrow$ clustering of data in space defined by W_{cur} .
- For each dimension i
 1. Updated $W_{cur} \leftarrow$ Increase or decrease the i -th dimension of W_{cur}
 2. $Cl_{cur} \leftarrow$ Cluster data in new space defined by W_{cur} .
 3. $Quality(W_{cur}) \leftarrow QoC_{constr}(Cl_{cur})$
 - If there is improvement to $Quality(W_{cur})$ Go to step 1
- $W_{best} \leftarrow$ weighting resulting in 'best' clustering (correspond to maximum $QoC_{constr}(Cl_{cur})$)



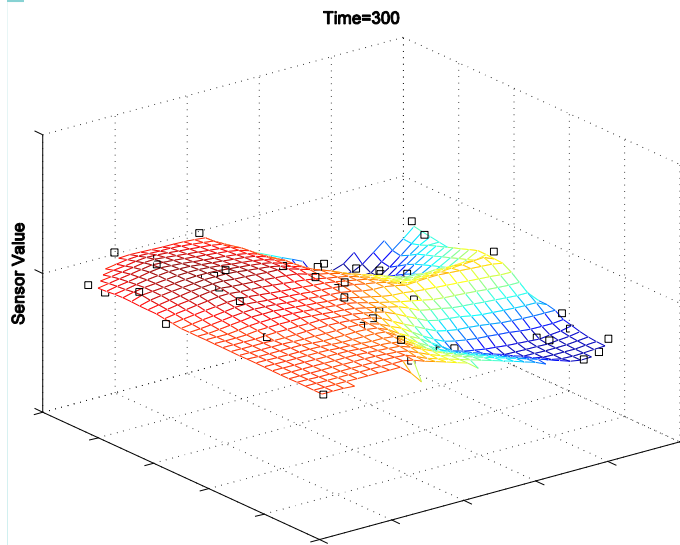
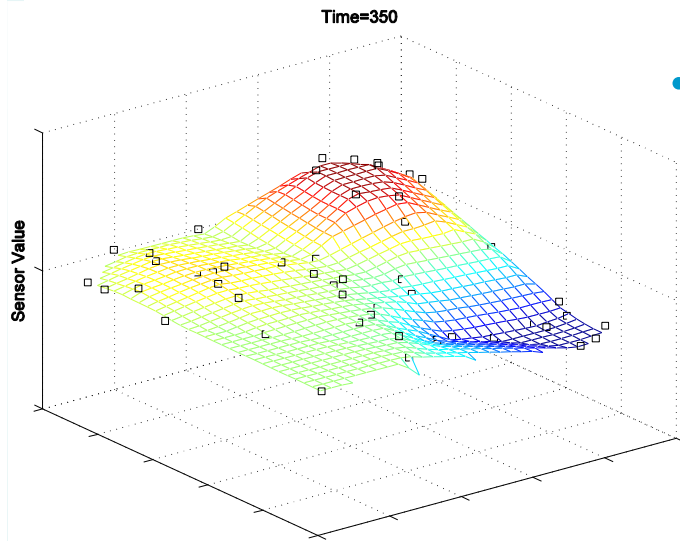
An application: Monitoring the state of a sensor network

- In many cases, aggregate information is sufficient
- It is important to conserve energy: minimize number of messages, data transmission

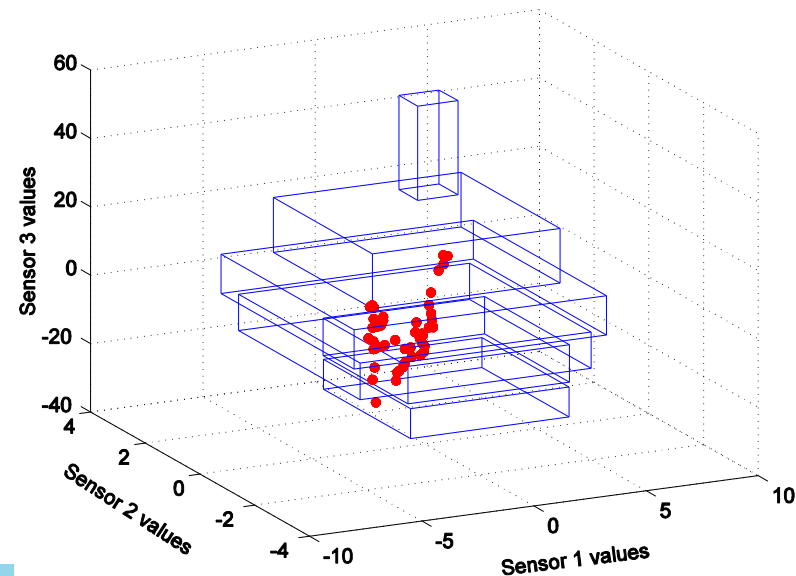
 The image cannot be displayed. Your computer may not have enough memory to open the image, or the image may have been corrupted. Restart your computer, and then open the file again. If the red x still appears, you may have to delete the image and then insert it again.

 The image cannot be displayed. Your computer may not have enough memory to open the image, or the image may have been corrupted. Restart your computer, and then open the file again. If the red x still appears, you may have to delete the image and then insert it again.

Abstracting the state of the network

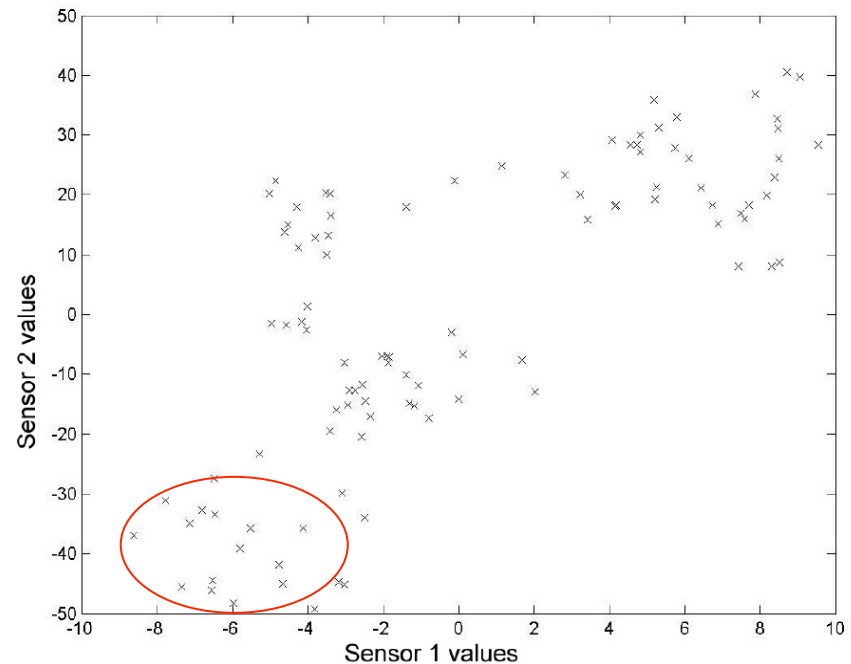


- Identifying a set of states the network can be in, and finding descriptions for the states, allows more efficient monitoring and intuitive results



An iterative clustering with user constraints algorithm identifies the states

- Network configuration:
The value of the sensors at a time t_i
- State:
A description of frequent similar configurations



Summary

- **Clustering with constraints**
 - Appealing, well-motivated problem
 - Significant recent work
 - Many problems are open:
 - How to properly integrate the constraints
 - Hierarchical algorithms
 - Convergence of iterative algorithms
 - How to find non-linear embeddings that respect the constraints
 - Kernel methods best current approach